



Terrawarden Drone Cleanup

A Major Qualifying Project
submitted to the Faculty of
Worcester Polytechnic Institute
in partial fulfillment of the requirements for the
degree of Bachelor of Science

Submitted by:

Mark Caleca
Zephyr Conley
Jakub Jandus
Samuel Markwick
Kevin Siegall
S. Taylor

May 2025

Submitted to:

Professor Berk Calli
Professor Kevin Leahy
Professor Guanrui Li



WPI

This report represents the work of one or more WPI students submitted to the faculty as evidence of completion of a degree requirement. WPI routinely publishes these reports on the web without editorial or peer review.

Abstract

Roadside trash on highway medians causes significant environmental and economic harm in the United States. While prevention and public awareness remain the best long-term solutions, current methods of cleaning up existing litter are ineffective, dangerous, and expensive. Although ground robots have been successful at collecting trash in other conditions, the inaccessible and uneven terrain of highway medians is better suited for an autonomous aerial system. The Terrawarden drone, a one-meter-wide quadrotor with a 3-DOF manipulator and compliant gripper, addresses the problem of roadside litter by autonomously detecting and retrieving trash using on-board perception. To validate the feasibility of this approach, a controlled experiment was designed in which the drone was tasked with detecting a beverage container, navigating to it, collecting it without landing, and then transporting it to a designated collection point. The drone successfully completed each of these tasks, demonstrating the effectiveness of the proposed system and its potential for future research and deployment.

Acknowledgments

The Terrawarden team wishes to acknowledge and thank our project advisors: Professor Berk Calli (RBE/CS/ME), Professor Kevin Leahy (RBE/CS), and Professor Guanrui Li (RBE) for their unwavering support, expertise, and mentorship throughout this project. Their insightful input, encouragement, and dedication have been indispensable to our progress and success. We would also like to thank Professor Melissa Butler for her invaluable feedback and writing advice, which was crucial to our paper's success.

Table of Contents

1	Introduction	4
2	Background	5
2.1	Roadside Litter	5
2.2	Automating Litter Collection	6
2.3	Types of UAVs	10
2.4	Aerial Manipulation	11
2.5	Aerial Manipulators for Trash Collection	14
2.6	Project Statement	15
3	Methodology and Iteration	16
3.1	Drone Platform	17
3.2	Software Architecture	21
3.3	Detection Model Training	24
3.4	Manipulator Design	30
3.5	Vision-Based Manipulation (VBM)	39
3.6	System Integration	44
4	Results and Experimental Evaluation	47
4.1	Drone Performance	47
4.2	Detection Model Validation	49
4.3	VBM Evaluation	51
4.4	Arm Performance	52
4.5	Gripper Performance	54
4.6	Integrated System Evaluation	56
5	Limitations and Implications	61
6	Conclusion	62
7	Appendices	63
A	Open-Source Can Models	63
B	Arm Inverse Kinematics	63
C	Drone Flying Safety Checklist	65
D	Results Videos	66
E	Open-Source Code Repositories	66
F	Contributions	66

1 Introduction

Despite national cleanup efforts, nearly 50 billion pieces of litter exist on American roadsides and waterways [7]. This trash, comprised mainly of fast-food packages, drink containers, plastic bags, and shredded tires, can cause issues such as traffic delays, vehicle collisions, and environmental damage [41].

Although methods for removing roadside litter already exist, they are expensive, dangerous, ineffective, and at times unethical [41]. Public awareness and preventative legislation, such as bottle deposit laws, are the best methods of decreasing the rate of litter accumulation, but do not address removing existing trash [7]. Instead, the removal of litter from highway medians is handled by volunteer organizations and prison labor. These workers are required to handle hazardous materials such as sharp objects, medical waste, and items contaminated by bodily fluids and viruses [41]. They also are at a much higher risk of getting seriously injured if there is a car accident near the work site [38].

A potential solution to the ethical, safety, and environmental concerns associated with removing roadside waste is automating the process with a robotic solution. Aerial manipulation, a new field of robotics that involves attaching robotic arms to Unmanned Aerial Vehicles (UAVs), is a novel approach to trash collection [40]. UAVs are advantageous for trash collection because they are cost-effective, can cover difficult terrain, and are easier to deploy in and out of a cleanup area on demand when compared to other robotic systems [89]. Due to their ability to cover a large area quickly, UAVs can utilize onboard image processing to identify trash, collect it, and deliver it to the ground station much quicker than traditional ground robot systems [30]. While collecting trash, a UAV can store data on the most common types of litter and where it is concentrated along the highway. This information that can be used to study how litter accumulates on highway medians and improve the effectiveness of preventative measures.

The goal of this project is to evaluate the feasibility of using an aerial manipulator for trash collection on highway medians by evaluating its performance in a testing environment. To simplify the problem and define clear constraints, the prototype was tested in an indoor drone cage and highway litter was represented by soda cans of various shapes and weights. The following objectives were proposed to evaluate the drone's success:

- Search a pre-defined area and detect a can
- Navigate to the can and successfully grasp it
- Return to a drop-off location and deposit the can

Through in-depth research, aerial manipulation was established as a viable means of addressing the roadside litter issue in the United States. Multiple manipulator arms, end-effector grippers, and vision systems were designed and evaluated based on payload capacity and real-time performance. The best performing arm, gripper, and vision systems were then integrated into a prototype consisting of a multicopter UAV base, autopilot system, onboard computer running the selected vision model, manipulator, and a feedback controller that coordinated aerial positioning and manipulation. Ultimately, this prototype was a successful proof-of-concept for the use of aerial manipulators for the detection and collection of roadside trash.

2 Background

2.1 Roadside Litter

Current State of Roadside Trash and Common Roadside Trash

Litter on highways, parkways, and other roads is a growing problem around the world. As more vehicles are used on roads, increasing amounts of trash is generated on roadsides [41]. According to a study conducted by Keep America Beautiful in 2020, nearly 50 billion pieces of litter lie along roadways in the United States[7]. Approximately 88% of this litter is four inches in size or smaller, and generally consists of cigarette butts, plastic film, food-packaging, drink containers (i.e. soda cans, water bottles, and plastic straws), construction debris, and personal-protective equipment [7].

Drawbacks of Current Methods of Trash Collection

Although significant progress has been made to reduce litter on U.S. Roadways, reducing the amount of visible trash by 54% since 2009, it is still very common to see litter along American roadsides and waterways [7]. While legislation such as bottle deposit laws decrease the future accumulation of litter, removal of existing trash falls mainly on volunteer organizations and prison labor [7]. These cleanup efforts have a large impact on local communities affected by roadside trash, but they do not happen very often. Even volunteer groups for large cleanup programs such as Adopt-A-Highway (AAH) and Sponsor-A-Highway (SAH) only manage to clean a small (one to three mile) section of a road two to four times a year [41].

Furthermore, roadside litter often consists of hazardous materials such as sharp objects, medical waste, and items contaminated by bodily fluids and viruses [41]. While programs like AAH and SAH work to educate and protect their volunteers, trash collection still puts them at risk for injury, illness, and infection [41]. Part of this risk involves transportation to and from the highway median, which puts workers and motorists at a much higher risk of vehicle collision [59] [29]. In an analysis from 2016-2018, the transportation data analysis firm Impact Research found that half the deaths and almost 1 in 5 serious injuries occur when a vehicle strikes a pedestrian who is leaving, working on, or returning to a stopped vehicle [38]. Drivers also have a tendency to slow down and stare when they see something unusual on the side of the road, such as an accident or a group of people gathering trash on the median [51]. This phenomenon, known as rubbernecking, causes about 35% of vehicle crashes on US highways [51]. To protect the safety of drivers and volunteers, it is essential to find alternative methods of trash cleanup for high-risk areas like highway medians.

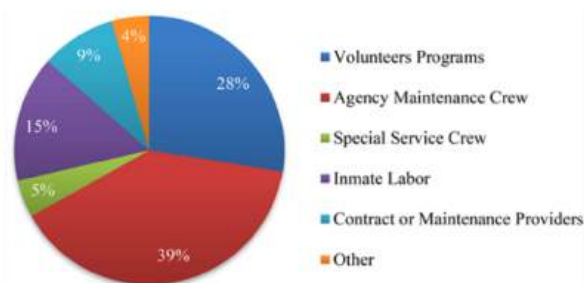


Figure 1: Distribution of trash cleanup labor [41]

There are also some moral issues with the way trash cleanup is currently handled. As shown in Figure 1, 15% of all highway trash collection is done by inmate laborers [41]. These prisoners cannot refuse to work, choose what their work is, or negotiate their compensation [16]. Most prisoners are paid nothing at all, but even those who do receive compensation are paid a fraction of minimum wage [16]. In California, both professional firefighters and prisoners fight wildfires, but professionals earn about \$74,000 annually compared to the \$1.45 a day (\$529.25 a year) that prisoners earn [31]. While this is technically legal under the Thirteenth Amendment, many consider it unjust and a gross human rights violation [79, 31]. Despite inmates receiving little to no compensation for their labor, the cost of transporting and supervising work

crews makes prison labor extremely expensive. Massachusetts, for example, spends approximately one million dollars per year on its Inmate Labor program [8].

Overall, although preventing further accumulation of trash and increasing awareness about the impact of litter are by far the best methods for eliminating future roadside litter [7], current solutions for removing existing trash are costly, dangerous, and in some cases immoral. Finding an ethical, cost-effective, and safe way of collecting trash on highway medians is therefore an essential engineering research priority.

2.2 Automating Litter Collection

Robots are often used to automate repetitive tasks, and tasks that are unsafe for humans to do. While robotic solutions for removing roadside litter are a newer concept, automation has existed in the janitorial industry for around 30 years [62]. Examples of existing automation techniques include using computer vision to identify and categorize trash and household cleaning robots like Roombas [48, 32]

Computer Vision for Litter Detection

Modern approaches for automated litter detection utilize machine learning for computer vision in the form of Convolutional Neural Networks (CNNs), a category of deep learning algorithm optimized for image processing. CNNs recognize objects within an image by breaking the image down into important characteristics, or features, then identifying the pattern of features within the image that correspond to the desired object [56]. Computer vision (CV) is a very broad field, and CNNs can be trained for a variety of tasks including classification (e.g. categorizing animals in an image as either a dog and a cat, Figure 2), segmentation (identifying what part of the image depicts an object, Figure 3), or detection (drawing a box around the rough outline of specific objects in an image), Figure 4 [33].



Figure 2: Classification example of an image of a dog



Figure 3: Segmentation example of a drone racing gate



Figure 4: Detection example of bottles on a field [44]

Machine Learning is built on the idea of replication, of models reproducing the process needed to turn an input into a wanted output. By modeling a mathematical pipeline with many parameters (called weights), these weights can be gradually adjusted to make the model more accurately process any set of training data into the desired output, assuming the model was given enough examples to properly learn [60]. In particular, CNNs are made up of many layers of convolutions, a mathematical operation akin to slowly scanning across the image with a lens, called a kernel, which exaggerates features in certain portions of the lens (see Figure 5) [56]. An example of this is the Sobol Kernel, a common convolutional kernel used for edge detection (See Figure 6). Through chaining together multiple convolution layers, a model can be configured to extract specific features or patterns within an image prior to recognition. Prior neural network methods needed weights to collate data across every input datapoint and across neurons, requiring exponentially more weights than CNNs. Each convolutional layer has just one kernel, which is usually a small square of weights (often 5x5, 7x7, or 9x9) which operates on the entire image. In this way, a much smaller set of weights are used to process very large input sizes (e.g. a 3-channel (RGB) 256x256 image is 196,608 input values). Using CNNs allows computer vision tasks to have orders of magnitude fewer weights to train when compared to traditional neural networks, drastically reducing the computation and training overhead needed to utilize these models [56].

Even so, accurately training CNNs requires vast amounts of data, often needing thousands or even tens of thousands of images to output accurate results [11]. Datasets of this magnitude can be difficult to compile, especially when training a model to identify dangerous or uncommon scenarios such as an airplane performing an emergency landing on a road or highway medians from the perspective of a drone [56]. Fortunately, many open-source datasets for training machine learning (ML) models exist online, even for niche applications [44].

Finding the perfect dataset for a specific application is an arduous process. Ideally, the dataset should encapsulate all facets of the expected inputs (also called the domain), such that even edge cases inputs are still well within the normal operating range of the trained model. Using this line of thought, a dataset must

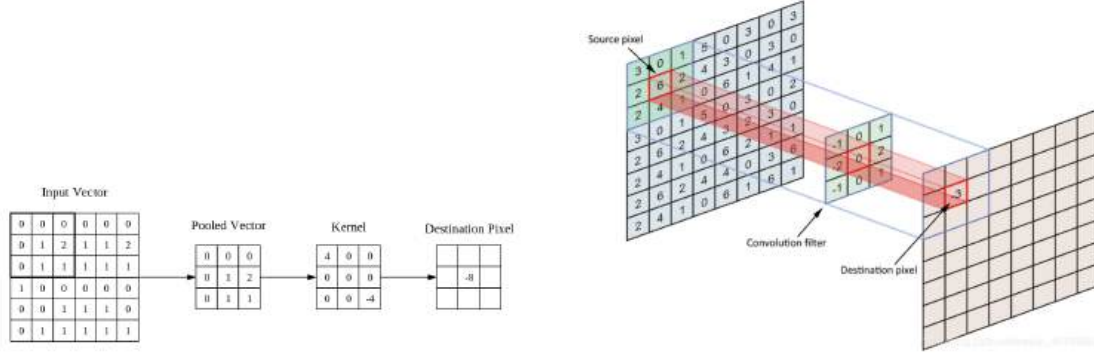
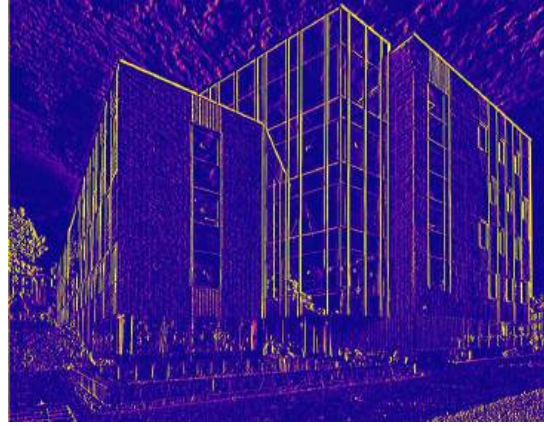


Figure 5: Convolutions process large matrices at once by applying a kernel to every pixel in the image



(a) WPI's Unity Hall



(b) Unity Hall, after Sobol Kernel processing

Figure 6: Sobol Kernel can be used for edge detection

contain many examples which are representative of what the model is going to see when applied, including the types of objects present, perspective from which the pictures are taken, and environments represented in the images [15]. Within the dataset, each image needs to have annotations, which specify where in an image a target object is. These are often stored in either COCO format (named after a dataset of the same name), or YOLO format (named after a model of the same name).

Trash Annotations in Context (TACO) is an open source annotated image dataset of waste in natural settings. From roadside litter in London to plastic washed up on tropical beaches, TACO aims to include diverse environments and types of trash [63]. Each image has annotations for waste items, with information on the category of trash, as well as information on the background, and if objects in the image are overlapping. Across 1500 images, this dataset has 60 categories of trash, which are split into 28 'super categories', which includes cans, bottles, plastic bags, etc[63]. There is also the UAVVaste dataset, which contains images taken of litter in natural environments from the point of view of an Unmanned Aerial Vehicle (UAV) [44].

Although open-source datasets are an extremely useful tool for training CV models, they are not always robust enough to train a model from the ground up. In these cases transfer learning, the concept of modifying a pre-trained model for a slightly different but related purpose, can be used to adapt an existing model to a niche purpose by fine-tuning it on a smaller dataset [35]. In the field of object detection, Common Objects in Context (COCO) is a large dataset often used to train object detection and segmentation models, and is a prime target for transfer learning [35]. COCO contains 330,000 images of which over 200,000 are fully annotated, and includes a large variety of environments, backgrounds, and object categories [48]. COCO is used very commonly for transfer learning additionally because the feature-extraction properties of CNNs share similar weights for most layers regardless of the object being detected [35]. Eden (2020) demonstrates

this by training a detection model for a trash identification & collection UAV on a custom dataset, starting with pretrained weights from a model trained on COCO [28].



(a) An image of a man on a bike with a dog, with a truck in the background.



(b) The image in Figure 7a, but with the COCO annotations shown.

Figure 7: A sample image from the COCO Dataset, shown with and without annotations.

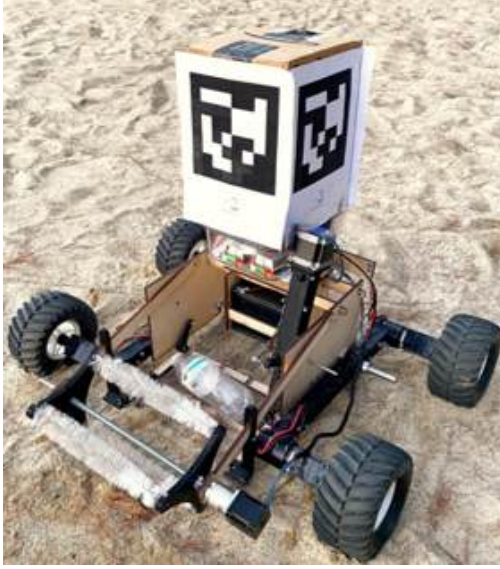
CNNs are highly versatile across different domains, making them a valuable tool for trash detection on highway medians. Using transfer learning, a base model trained to identify features in images can be adapted to recognize specific objects, such as roadside trash. As long as a robust dataset on the desired domain exists, a Convolutional Neural Network can be applied to any specific object detection task.

Automation of Litter Collection

Janitorial robotics is a growing field that includes robotic solutions for waste management, commercial cleaning services, and household cleaning. The robotic vacuum cleaner market alone, which includes products such as robotic vacuums and pool cleaners, is expected be worth \$31.8 billion by 2033, a 17% growth in the next eight years [32]. Companies such as iRobot have been developing household floor cleaning robots for decades, which use a combination of sensors to autonomously navigate an unknown environment [22]. Land-based environmental cleanup robots, in contrast, have yet to become commercialized in the same way as robotic vacuums.

Despite not being a large industry, the development of environmental cleanup robots has been a growing field of robotics research in the past few years. These robots remove litter from environments such as beaches, parks, and roads using unique devices for collecting trash [9, 10, 77, 78]. Environmental cleanup robots deployed in sandy areas, such as BeBot, often use a sieve mechanism that removes trash from the ground without disturbing the surrounding environment [9]. In addition to sieves, environmental cleanup robots often collect trash with a robotic manipulator because they can precisely pick-and-place objects. For instance, WPI’s BeachBots MQP, shown in Figure 8a uses a 2 degree-of-freedom (DOF) Revolute-Revolute (RR) manipulator to collect trash from beaches and deposit it into a collection bin [10]. Another example of an environmental cleanup robot with a robotic manipulator is WPI’s TrashBot MQP, shown in Figure 8b. This project focused on collecting trash from grassy fields by attaching a 4-DOF manipulator to a wheeled mobile robot, which improved the robot’s mobility and ability to grasp objects of different shapes, sizes, and orientations [77]. In addition to current research, environmental cleanup robots are beginning to be available commercially as well. Since 2011, Trombia Technologies has been developing street-sweeping robots designed to clean urban environments. These systems rely on sensor fusion, including LiDAR, computer vision, and GPS, to route optimized paths through urban environments while detecting roadside litter. It then collects trash from roads and sidewalks using a combination of street brushes and vacuum suction [78].

Ground-based robots with wheeled mobility have varying trade-offs between mechanical simplicity, adaptability, and dexterity. Robots like WPI’s BeachBots and TrashBot demonstrated precision in picking up individual pieces of trash with their manipulators, while sieving and brush-based methods attempt to pick



(a) WPI MQP BeachBot collected trash along beaches with a 2-DOF RR manipulator [10].



(b) WPI MQP TrashBot utilized a 4-DOF manipulator to collect trash in grassy fields [77].

Figure 8: Different WPI MQPs have worked to collect trash in several different environments using sensors and a manipulator.

up more litter faster with potential for missing some pieces of trash. These different implementations provide inspiration for how robotic, sensor-based approaches can be used in robotics to collect trash.

2.3 Types of UAVs

Aerial robots, broadly falling under (and often mislabeled as) drones or UAVs, are a novel approach to litter collection in outdoor environments. As airborne vehicles, they can scout a lot of area in a short period of time, but are limited. However, their construction is lightweight stemming from the limited lifting capacity, which also influences how much trash can be picked collected in a single flight.

UAVs are commonly classified by their lift and propulsion architectures into fixed-wing, rotor-wing (helicopter or multicopter) [37]. Less frequent UAV types include hybrid (vertical take-off & landing fixed winged aircraft that have four or more propellers for take-off and landing and fixed wings for long-range flight) and lighter-than-air systems (balloons and airships) [61]. Fixed-wing UAVs consist of a fuselage, rigid wings, propulsion unit and control surfaces; they excel in forward-flight efficiency, range and endurance but cannot hover or reverse [53]. Rotor-wing UAVs can be further split into single and multi rotor designs. Single-rotor UAVs – essentially unmanned helicopters – use one main rotor and a tail rotor to achieve vertical take-off, landing and stable hover; their mechanical complexity and vibration sensitivity require skilled piloting and maintenance, but offer superior efficiency to their multi-rotor counterpart. Multi-rotor UAVs are the most cost-effective and straightforward to build, coming in configurations such as tricopters, quadcopters, hexacopters, and octocopters. Quadcopters, in particular, have become popular thanks to their vertical takeoff and landing, good maneuverability, high agility, compact size, and simple, budget-friendly design [53].

Quadcopters

Quadcopters, a popular type of multi-rotor small UAV, exemplify these gains and trade-offs. [61]. One of the major advantages of quadcopters is their simplified design [37]. Unlike traditional helicopters, they do not require the complex swashplate mechanisms for rotor control. “In aeronautics, a swashplate is a mechanical device that translates input via the helicopter flight controls into motion of the main rotor blades. Because the main rotor blades are spinning, the swashplate is used to transmit three of the pilot’s commands from the non-rotating fuselage to the rotating rotor hub and mainblades.” [18] Instead, they rely on the differential

thrust of four fixed-pitch propellers, which simplifies development and reduces maintenance requirements [37, p. 6]. Their modular nature also allows for easy customization with different sensors, cameras, and payloads. Additionally, while quadcopters have limited redundancy (a failure in one rotor can lead to loss of control), designs with more rotors, such as hexacopters and octocopters, offer improved fault tolerance for safer flight operations and are usually larger, thus yielding more payload capacity [6] [61]. Finally, due to their small size and fewer rotors, quadcopters are more susceptible to wind disturbances and adverse weather conditions in outdoor environments than their larger counterparts [61].

Beyond operational challenges, drones face technical and regulatory constraints. Quadcopters are inherently underactuated, having only four independent actuators producing thrust along the quadcopters' body-fixed vertical axis. These four motors control directly the Z-axis motion and, through differential speeds, create torques that influence roll, pitch, and yaw [37, pp. 6–9]. The horizontal motions along the X and Y axes cannot be controlled directly. Instead, the quadcopter must tilt its entire body (by adjusting roll and pitch) to redirect a portion of the vertical thrust into horizontal movement. Multicopter's reliance on thrust for both lift and maneuvering limits payload capacity compared to fixed-wing UAVs or ground-based robots, though this is less of a drawback for tasks like picking up small pieces of litter scattered across large areas. Flight time is usually limited to tens of minutes as heavy batteries have to be carried and significant energy is expended just to maintain altitude, an issue that ground robots do not face. [61]

The operation of drones comes with a lot of restrictions, including strict regulations on where, how, and which UAVs can fly, especially over crowds of people or near other aircraft [46]. These rules ensure operational safety, but combined with a small set of favorable flying conditions (low wind, high visibility) limit where drones can be deployed [1].

2.4 Aerial Manipulation

Types of Aerial Manipulators

To enable the proposed UAV to not only observe but also physically interact with objects during its missions, the choice of aerial manipulator is critical. Aerial manipulation is a relatively new field within robotics that integrates UAVs with robotic arms to enable physical interaction with the environment [40]. This capability extends the traditional UAV applications beyond passive observation to active grasping, manipulation, or interaction with objects. Aerial manipulators have been proven capable of completing simple pick-and-place options [57]. However, there is a variety of manipulators that have been applied to this type of system. The most basic aerial manipulator is known as a 'Flying Hand'. In many cases, these are systems where a rigid tool, like a gripper, is directly attached to a UAV. These lack dexterity, and are often lightweight, low-complexity systems [71].

Some aerial systems integrate actuated robotic arms to enable more versatile interaction with their environment. A key design consideration is the number of DOFs, with most actuated robotic arms having at least 3 DOF. Within the category of actuated robotic arms, there are a number of various actuators and joint configurations that can be combined to create unique systems for specific purposes. One example is the AIRCRAB, a hybrid system combining a quadrotor, a 3-DOF arm, and a stabilizing ground-contact wheel for object manipulation [13]. Another approach involves cable-actuated manipulators, such as those proposed by AlAkhras et al. and Xu et al., which keep heavy motors stationary using a system of cables and remote actuators, reducing shifts in the system's center of mass [2, 86]. As more DOFs are added, payload control can be more precise [58]. Today, many aerial manipulators are highly redundant systems (>6-DOF), and sometimes even include two arms [58].

Computer Vision for Manipulation

Computer vision is a useful tool that enables an aerial manipulator to autonomously locate and grasp an object. By using cameras mounted on the manipulator or the robot, the robot can determine an object's location and orientation. Vision systems let robots plan the best way to approach an object by providing real-time feedback on the environment and the target. However, vision-based manipulation (VBM) is not always simple to implement, especially when the robot operates in dynamic environments or is grasping objects of various size, shape or material [74].

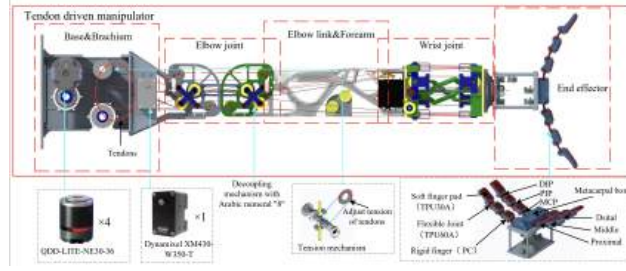


Figure 9: Example of a cable actuated arm for an aerial manipulator as seen in [86]

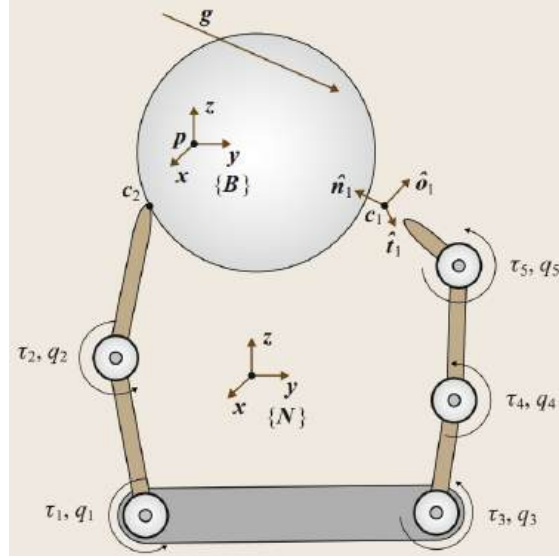


Figure 10: A robot manipulator grasps a spherical object at two contact points in 3D space, c_1 and c_2 [74].

When implemented, grasp planning for VBM typically begins by calculating the grasp matrix $\mathbf{G}$ and the hand Jacobian $\mathbf{J}$, mathematical models that describe the velocity kinematics and force transmission between the manipulator and the object at each point of contact [74]. Grasp matrices are constructed using the contact model (e.g. hard finger) and position of each contact point (shown as c_1 and c_2 in Figure 10) relative to the object's frame of reference $\mathbf{B}$ located at $\mathbf{p}$. They use this data to model how forces applied at these points generate a net wrench on the object. Contact modeling, a form of mechanical analysis, defines the range of forces that can be transmitted through each contact point as a function of the geometry and material properties of both the manipulator and the object; these forces are then used to calculate $\mathbf{G}$. Contact modeling establishes the physical limits and constraints of contact points, while the grasp matrix models how these forces generate wrenches on the object as a whole [74]. The full equations for calculating a grasp matrix $\mathbf{G}$ are shown in Equations 1, 2, 3, 4, and 5, where $\mathbf{P}_i$ is the contact point grasp transformation matrix for each contact point $\mathbf{c}_i$ relative to the object's centroid $\mathbf{o}$ and $\mathbf{R}_i$ is the rotation matrix between the local contact frame and the world frame.

Method	Training Size	Test Set	Attempts	Success Rate (%)
Proposed method	400	similar to GPP	80	97.50
		similar to DexNet2.0	90	97.78
		similar to GG-CNN	120	99.17
		Self-designed	180	98.33
		All objects	400	98.25

Figure 11: Grasp success rate of a feature detection deep learning model on datasets taken from similar studies [55].

$$\text{skew}(\mathbf{v}) = \begin{bmatrix} 0 & -v_z & v_y \\ v_z & 0 & -v_x \\ -v_y & v_x & 0 \end{bmatrix} \quad (1)$$

$$\mathbf{P}_i = \begin{bmatrix} \mathbf{I}_{3 \times 3} & \text{skew}(\mathbf{c}_i - \mathbf{p})^T \\ \mathbf{0}_{3 \times 3} & \mathbf{I}_{3 \times 3} \end{bmatrix} \quad (2)$$

$$\mathbf{P} = \begin{bmatrix} \mathbf{P}_1 \\ \vdots \\ \mathbf{P}_N \end{bmatrix} \quad (3)$$

$$\bar{\mathbf{R}}_i = \begin{bmatrix} \mathbf{R}_i & \mathbf{0} \\ \mathbf{0} & \mathbf{R}_i \end{bmatrix} \quad (4)$$

$$\mathbf{G} = [(\bar{\mathbf{R}}_1 \mathbf{P}_1)^T \quad \cdots \quad (\bar{\mathbf{R}}_N \mathbf{P}_N)^T] \quad (5)$$

During VBM, a graspable object is localized in 3D space using data from a depth sensor. This data often takes the form of a point cloud, a 3D map of points, each representing an (x, y, z) coordinate in the world frame. Potential grasp points are retrieved by filtering the point cloud with methods such as edge detection based on surface normal curvature [72]. The number of potential grasp points varies based on filtering method, physical characteristics of the manipulator and the object, and the robotic application of the VBM system [54]. The algorithm selects the best grasp point by calculating the grasp matrices for each potential point and ranking them according to quality measures, such as the extent to which the grasp conforms to the object’s geometry [36, 69]. However, depending on factors such as the number of sampled points, contact points between the manipulator and the object, and the gripper’s complexity, this process can be computationally intensive. Even simple VBM models in static environments require fine-tuned camera calibration and precise manipulator control [54].

Some modern approaches believe machine learning could reduce the computational load and reliance on camera and manipulator accuracy that grasping tasks currently require. For example, Morrison et al. [54] from the Australian Centre for Robotic Vision developed the Generative Grasping Convolution Neural Network (GG-CNN), which directly predicts the grasp pose — comprised of the manipulator’s position and orientation — and the associated grasp quality score for each pixel of the image. This model had a compute speed of 19 ms and successfully grasped household items in 88% of trials and objects with adversarial geometry 83% of the time. Khor et al. [42] also concluded that deep learning could be used for accurate, reliable grasping. They trained a feature detection model, which extracts the edges, corners, and centroid of an object, then picks the optimal grasp by identifying flat surfaces near the object’s center of mass. This model performed even better than Morrison et al. [54], reaching an average accuracy of 98.2% across datasets used in various grasp studies. The full accuracy breakdown across each dataset is shown in Figure 11 [55].

Although machine learning models have the potential to vastly outperform existing methods of grasp planning, they are currently not robust enough to adapt to a real-world environment [84]. Additionally, CNN models like GG-CNN are limited by the amount of data required to train them, which makes expanding them for broader applications very difficult [42]. GG-CNN trained using 51,100 data points, a notably small dataset for a grasp CNN, and was only evaluated on 12 household and 8 adversarial objects [54]. One study, which

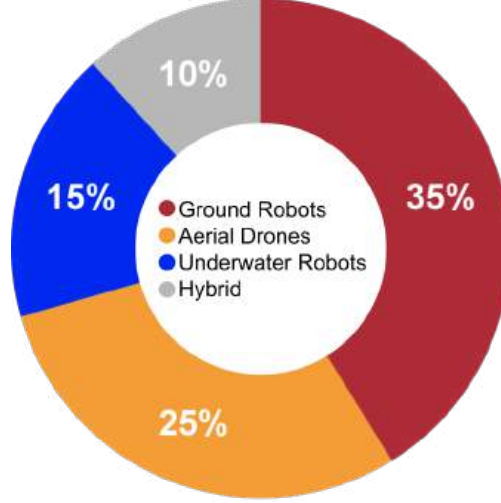


Figure 12: Types of robots used in studies involving hazardous waste [45]

compared the grasping performance of traditional 2D and 3D heuristic models to ML models found that the 3D heuristic outperformed ML, especially for objects that were harder to grasp. The heuristic models also generalized well to real-world testing, while the ML models were more sensitive to changes in sensor behavior [55]. Another study also compared how different grasping methods responded to noise altering an object’s position and orientation in a simulated environment. In this study, a robotic manipulator was tasked with grasping objects in a simulated environment where random noise altered the object’s position and orientation after being scanned by a camera. Four different methods, were used to calculate the optimal grasp: GG-CNN, Robust GG-CNN, contour grasp generation, and naïve centroid grasp generation. While the success rate of all planning methods decreased as the pose variance became more noisy, the centroid-based approach generally outperformed the neural network-based approaches in terms of reliability [12]. Similarly, a review by a group at the Agency for Science, Technology, and Research in Singapore in 2023 acknowledged that the success rate of “learning-based grasping pose generation...is still not reliable enough to be implemented in the real world” [84]. This evolving research highlights the ongoing challenges of consistently and reliably performing grasping tasks in dynamic, real-world environments, as well as the potential drawbacks of using neural networks in these settings. Although neural network-based approaches like GG-CNN show potential for grasping in dynamic environments, further development is needed before they can be used outside of a laboratory setting [84]. In situations with environmental noise and object variability, such as collecting trash with an aerial manipulator, sticking with traditional methods may be a more reliable version of grasp planning.

2.5 Aerial Manipulators for Trash Collection

Although ground robots are mainly used for trash collection, recent improvements to UAVs have led to UAVs being utilized for this task as well [89, 45]. UAVs are advantageous for trash collection because they are cost-effective, able to reach non-navigable spots, and easier to deploy in and out of a cleanup area on demand, especially when compared to existing robotic solutions [89]. In addition, drones are already commonly used to map and classify pollution in areas such as beaches with great success [30].

Traditional ground robots still pose some advantages over aerial manipulators. Ground robots are capable of carrying much larger payloads, are more stable overall, and have larger lifespans [45]. Shown in Figure 12, 35% of current studies on autonomous solutions for removing hazardous waste deploy ground robots for sample collection and cleanup. While drones were used in 25% of current research, they were mostly focused on surveillance and monitoring rather than collection [45]. However, the biggest drawback of ground robots is their inability to navigate complex and hard-to-reach terrain [45].

Since the main advantage of aerial manipulators compared to ground robots is their ability to navigate complex terrain, UAVs serve as an extremely useful tool for trash collection in unusual and hard to reach

environments such as water environments and highway medians [89]. UAVs also have an application in fragile environments such as beaches that can suffer immensely from the presence of a large and potentially destructive ground robot [30]. It is possible that drivers could get distracted if they spot an autonomous robot flying above the highway, potentially leading to hazardous driving conditions and traffic congestion. However, a large ground robot or parked transport vehicle are likely to cause similar effects. Increased maneuverability in uncommon terrain as well combined with improvements to payload capacity, battery life, stability, and downdraft, as well as swarm systems of aerial manipulators, allow UAVs to serve as a viable method for trash collection along highway medians.

2.6 Project Statement

The goal of this project is to provide proof-of-concept for the use of aerial manipulators for efficient litter collection. The system will be manually deployed via a drone that takes off and navigates within a designated cleaning zone. Once airborne, the drone will execute a systematic search pattern to locate pieces of litter. Upon detection, the drone will descend, deploy its manipulator arm to securely grasp the trash, and then return to a designated deposit point where the litter can be offloaded. This approach aims to enhance highway cleaning efforts by combining manual oversight with automated collection capabilities. To fit within the project timeline, a limited scope was defined. Instead of producing a system capable of functioning in highway medians, this project aims to provide proof-of-concept for the use of aerial manipulators for trash collection, especially in hard-to-access areas. The goal of this project is to create a prototype capable of identifying and collecting trash in a secure, indoor environment with no aerial obstacles. To simplify the requirements of the detection and grasping subsystems, trash is represented by standard soda cans. By creating a successful proof-of-concept prototype, this project will serve as a basis for future development that will transition the concept to a real-world commercial product.

3 Methodology and Iteration

The constraints of this project define a prototype that is capable of detecting, navigating to, grasping, then depositing litter at a designated location. For further clarity, it was decided that all testing would take place in a 4 m by 10 m drone cage, a workspace with padded floors and safety nets designed for indoor UAV testing. To fit the scope of the project, litter was represented by a mix of standard sized 12 oz soda cans and 8 oz Red Bull cans. The cans had a height range of 5-7 cm and were empty, partially full, or full, giving a payload weight range of 15-400 g. To provide a clear baseline of the project parameters and minimum viable product, a design specification that detailed the hardware requirements and desired performance of each subsystem was compiled. This is shown in Table 1.

Prototype Component	Parameter	Desired Value
UAV	Payload Capacity	2 kg
UAV	Size	1 m diagonal footprint from propeller tip to propeller tip
UAV	Position Accuracy	Able to hold position within manipulator workspace without GPS input
UAV	Flight Time	15-20 minutes per battery charge
UAV	Transportable	Able to be packaged and transported by a single person
Manual Controller	Emergency Response	Prevent catastrophic crashes
Autopilot	System Communication	Compatible with ROS
Autopilot	Modularity	Flexible customization and licensing
Autopilot	Documentation	Comprehensive technical documentation
Flight Computer	Computation Capability	Able to run all systems in real-time
Flight Computer	System Communication	Compatible with ROS
On-board Computation	Processing Speed	30 fps (matching camera frame rate)
On-board Computation	Computation Capability	Able to run all systems in real-time
On-board Computation	System Communication	Compatible with ROS
Vision Hardware	Depth Perception	Able to identify where an object is in 3D space
Vision Hardware	System Communication	Compatible with ROS
Vision Hardware	FPS	30 fps
VBM Software	Recall	Able to transpose a point from a 2D camera image to an (x, y, z) coordinate in the world-frame
Manipulator	Weight	Combined weight of arm and gripper < 0.5 kg
Manipulator	Payload Capacity	400 g
Manipulator	Max Workspace/Reach	28.4 cm below the carrying plate and 47.1 cm from the drone's COM
Gripper	Span	5-7 cm
Gripper	Grasp Security	Able to maintain grasp on object while UAV is in motion
Detection Model	Precision	False positive rate of <10%,
Detection Model	Recall	True positive rate of >60%,
Detection Model	Robustness	Able to detect cans located 0.25-6 m from the camera from different angles, camera perspectives, and lighting conditions

Table 1: Design parameters for the aerial manipulator prototype

This design specification simplified prototype development and created a clear metric for evaluating prototype success.

3.1 Drone Platform

To simplify the design process and create a modular platform, a preexisting aerial vehicle was chosen for the aerial manipulator base. Given the project scope, only multi-rotor platforms were considered as fixed-wing UAVs rely on forward motion for lift generation and single-rotor helicopters are unsafe to operate at low heights since their tail rotor could collide with the ground. The preliminary design parameters specified that the drone be easily portable, have a maximum wingspan of $1\text{ m} \times 1\text{ m}$ (across the diagonal propeller tips) and have a payload capacity of approximately 2 kg (without the battery). Purchasing a drone would have required most of the project budget, so an attempt was made to source a drone from an on-campus lab or faculty member. Ultimately two potential UAVs, both made by UAV Systems, were acquired: a quadcopter with a payload capacity of 1.5 kg and motor-to-motor diagonal of 0.67 m and an octocopter with a motor-to-motor span of exceeding 2 m. As the octocopter was nearly twice the size of the design parameters, it was not considered a viable option for this project. On the other hand, although the quadcopter’s diagonal propeller wingspan of 1.05 m and payload capacity of 1.5 kg were slightly outside the desired specification, the quadcopter could be used at no cost and its folding-manipulator design and carrying case made it easy to transport. By comparison, purchasing an off-the-shelf Holybro X650 Development Kit (with battery), a model that met all design parameters, would have cost about \$1,100 – nearly 75% of the total project budget. Ultimately, selecting the UAV Systems quadcopter allowed more budget to be allocated to actuators and high-performance onboard computation.

Initial Platform Viability

While the UAV Systems quadcopter was in very good physical shape, its lack of documentation made understanding and modifying its components and structural design extremely challenging. The only official documentation that came with the drone was a physical label claiming the maximum rated payload was 1.5 kg (excluding the battery). Although the UAV Systems webpage still existed, this specific model was no longer listed as one of their products. An extremely similar product, assumed to be a newer model of this quadcopter, was listed, but its design was owned by a different company, Aurelia Aerospace. Multiple attempts were made to contact both UAV Systems and Aurelia Aerospace to gain information about the old quadcopter design, but all attempts were unsuccessful. As the drone lacked component specifications and wiring schematics, a complete disassembly and reassembly of the system was required to understand its components and structural design.

During this process, the drone’s structure was modeled in SolidWorks, with emphasis on mounting and cable routing holes. Analysis of the drone’s carbon-fiber frame and plate-and-rods payload mounting system, shown in Figure 13, confirmed that they were in good mechanical condition. The propulsion system – consisting of four 40 Amp matching electronic speed controllers (ESCs), four MAD 4014 370KV brushless motors, four 38 cm long, two-bladed, foldable propellers, and a single 6S (six lithium-ion cells in series) 5000 mAh battery – was also determined to be suitable for the project. This decision was based on confidence in the manufacturer’s rated payload and component selection, as motors and ESCs are relatively standardized.

Drone Peripherals and Electronics Upgrades

As the overall code workflow and function development process depended on the autopilot software, finding firmware that met the design parameters was crucial. The autopilot needed comprehensive technical documentation, native ROS2 (explained in Section 3.2) support, and flexible customization and licensing. Research on popular autopilot software determined that the PX4, Ardupilot, and Betaflight all met the project needs. Ultimately, the PX4 autopilot platform was chosen based on its superior documentation, seamless ROS2 integration through a provided middleware package, and broader system customization capabilities under a permissive license [65]. To improve position tracking performance in areas with limited GPS availability and enable manual remote control of the drone with system status telemetry, a custom version of the firmware was compiled. Starting from the standard firmware (v1.14), ExpressLRS (ELRS) radio communication and an onboard optical-flow sensor were added. ELRS radio communication added long-range, low-latency manual control and a telemetry link between the remote control and the drone. Integrating the optical-flow sensor improved ground-proximity position tracking and interpolation between GPS position updates. The autopilot hardware was flashed to integrate the custom firmware.



Figure 13: A view of the original drone frame and carrier plate



Figure 14: 3D render of the CAD model of the drone

Additional changes included adding toggles to the remote control transmitter's switch layout for the following processes: position-hold, altitude-hold, landing, switching to offboard mode, arming the drone, and the kill function. The transmitter's telemetry display was updated to show battery percentage, remote control link strength, and current GPS coordinates (available when flying outdoors) to improve operator awareness. To evaluate the drone's flight performance and payload capacity, a manual hover test with a payload of 1 kg was performed. While hovering, the motors operated at less than 50% of their maximum thrust, indicating that the drone could handle the requirements of the project scope.

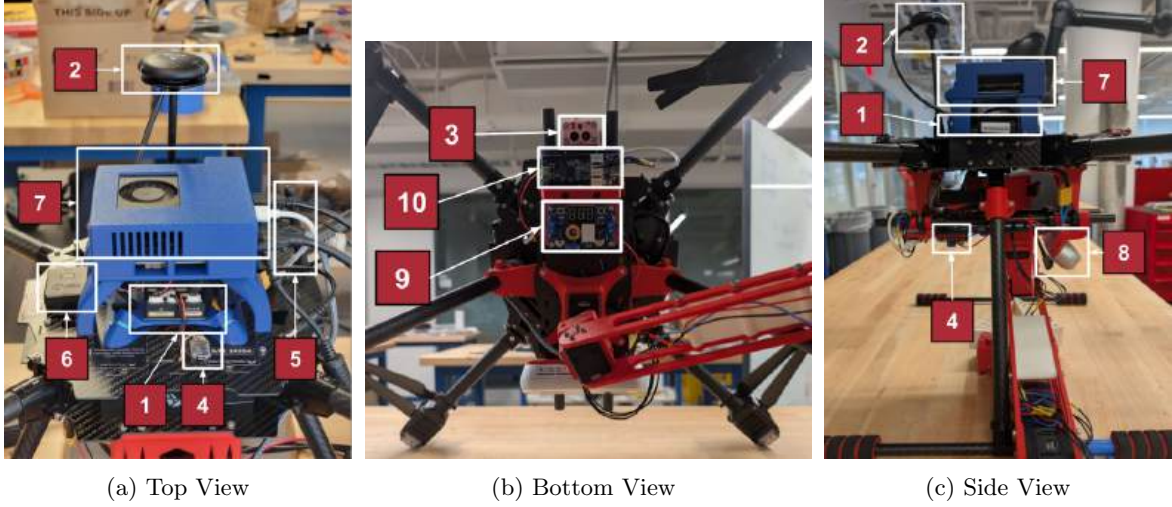


Figure 15: The drone, with its onboard components annotated.

Figure 15 number	Peripheral	Module / Part No.
1	Flight Computer	Holybro Pixhawk 6X v2A
2	Flight Computer GPS	Holybro M10 GPS
3	External Optical Flow and Distance Sensor	MicoAir MTF-01
4	Remote Control Receiver	ExpressLRS 2.4GHz Nano Receiver
5	Ground Station Telemetry	915 MHz Telemetry Radio
6	FAA Remote ID	Holy Stone Drone Remote ID Module
7	Companion Computer	Jetson Orin NX
8	Depth Camera	Intel RealSense D455
9	Power Step Down Board	XL4015E1 5A Buck Converter
10	Servo Control Board	Dynamixel OpenRB-150

Table 2: Integrated drone peripherals used in the multi-sensor architecture

As development progressed, the onboard flight computer, an AUV X2.1, began exhibiting unpredictable behavior. This flight computer suffered from RAM limitations and frequent telemetry dropouts when attempting to integrate the downward-facing optical flow sensor with the PX4 software. This was likely because the system was eight years old and was not built to handle this heavy of a computational load. These issues made the existing hardware unsuitable for launching μ XRCE-DDS, the ROS2 communication bridge service needed for the communication between the flight computer and the companion computer [64].

As a result, the AUAV X2.1 was replaced with a Holybro Pixhawk 6X flight computer. This autopilot computer was chosen because it was readily available and its recent development made it compatible with current technologies and protocols. To integrate the new hardware into the drone, the frame was disassembled for a second time and the internal systems were rewired to adapt the current and voltage sensors from analog to digital. As the Pixhawk was larger and heavier than the previous flight computer, a new vibration-isolating mount, shown in Figure 16, was designed to protect it during flight. During reassembly, the new mount and Holybro M10 GPS unit were installed on the drone to ensure system compatibility and maintain overall upgrade consistency. The sensor positions were defined in the configuration file relative to the front-right-down (FRD) local frame, with the origin set at the Pixhawk flight computer, as this is the local frame the PX4 firmware uses. Position references in the world frame are invariant and pass in the north-east-down (NED) frame. This is standard for PX4 and the rotation of this frame is magnetically locked to the Earth’s magnetic field, with the origin being placed at the drone takeoff position each flight.

In addition to the flight computer, another on-board computer was required for image detection, trajectory planning, and stereo vision processing. After evaluating the real-time computational power needed to



Figure 16: Pixhawk 6X flight computer on the vibration-isolating mount

simultaneously run an deep learning detection model and ROS2, the NVIDIA Jetson Orin NX was chosen as it is commonly used to run deep learning algorithms on mobile robots, has a base area of $10\text{ cm} \times 10\text{ cm}$, and weighs only 180 g. To shield the companion computer from potential damage in a crash, a sturdy protective case mounted above the flight controller, shown in Figure 17, was designed.



Figure 17: Jetson Orin NX in the protective mount

To concentrate weight around the drone’s horizontal center axis (shown in Figure 18) and thus reduce the torque on the drone actuators, the battery was positioned directly below the flight computer and secured using Velcro straps. The original drone frame did not have a battery tray, so a custom mount was designed, prioritizing balanced weight distribution and battery security. The final design of this mount is shown in Figure 18. The original vibration isolation system, composed of rubber damping rings and carbon-fiber rods, was attached to the battery mount using 3D-printed extension plates to improve payload stability.

Drone Electronics

Once the drone met the hardware requirements for the project, the middleware for the ROS2 communication bridge was set up. This required installing ROS2 on the Jetson and configuring the Pixhawk to automatically start communicating with it. The two systems communicate over a 100 Mb/s Ethernet link, a speed that prevents data transmission from being a limiting factor. Following completion of this setup and preliminary tuning of the autopilot parameters, the system successfully completed a second indoor hover test. During

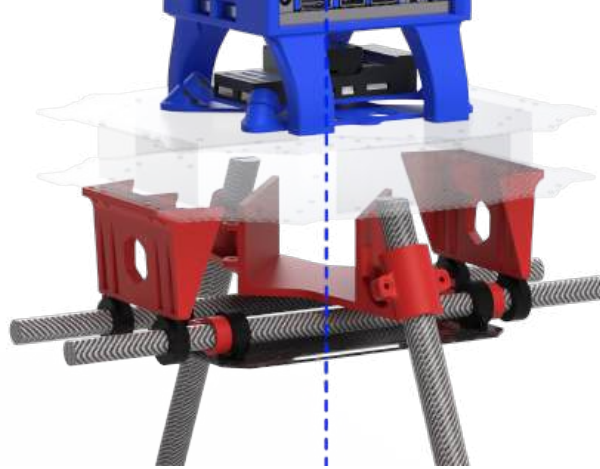


Figure 18: Custom designed battery holder (red, middle) and payload carriage (black, bottom). Drone frame Z-axis of rotation is represented by the vertical blue dashed line.

this test, the Jetson autonomously controlled the drone through the Pixhawk and held position with an estimated positional error of less than 50 centimeters without GPS signal. Over roughly twenty flights – including hover-only trials, manual-pilot sessions, and basic autonomous runs – the flight computer’s internal model-predictive controller was tuned for stable indoor operation. Originally, the Extended Kalman Filter (EKF) overestimated the noise from the optical-flow and downward-facing distance sensors causing the filter to rely too heavily on other metrics such as barometric pressure (which drifts near the ground), accelerometer approximation (which becomes more inaccurate over time), and GPS (which is unavailable indoors). Improving the noise estimate greatly improved flight performance, decreasing the positional error while hovering to less than 20 cm indoors. The PX4 firmware’s limits for maximum velocity, horizontal and vertical accelerations, and jerk were incrementally lowered to the lowest values the system would accept, improving maneuverability in the constrained, indoor testing environment. After this tuning cycle, the drone could hold its position and smoothly navigate around the flight cage with accurate, precise position and altitude control and exhibited minimal overshoot or oscillation. As the drone was now flight worthy, it was fitted with an external Remote ID module and registered as the Terrawarden Aerial Manipulator to comply with outdoor flight FAA regulations.

3.2 Software Architecture

To ease the development process, the software for the project was split into three main subsystems: drone control, object detection, and manipulator control. These subsystems all had different design requirements, dependent on the backing hardware, and required a singular architecture to control and manage the system. This core architecture needed to be able to control the flow of events, facilitate communication between the different subsystems, and run multiple processes simultaneously.

ROS2 Framework

The Robot Operating System (ROS) is an open source software development kit used for robotic applications in academia, research, and industry. It is designed to easily integrate with other software stacks and provides a communication system based on the Observer Design Pattern that manages communication between simultaneous isolated processes [67]. In ROS, each of these processes, named nodes, can be written in C++ or Python via the rclcpp or rclpy libraries (in ROS2). Each node can access the messaging system through the creation of publishers, subscribers, or services. Publishers send messages on a specific topic, which subscribers listen to and run a callback function upon receiving a message. Services are call-and-response requests, where a node calls a service and either waits for a response or receives a separate message once the service call is completed.

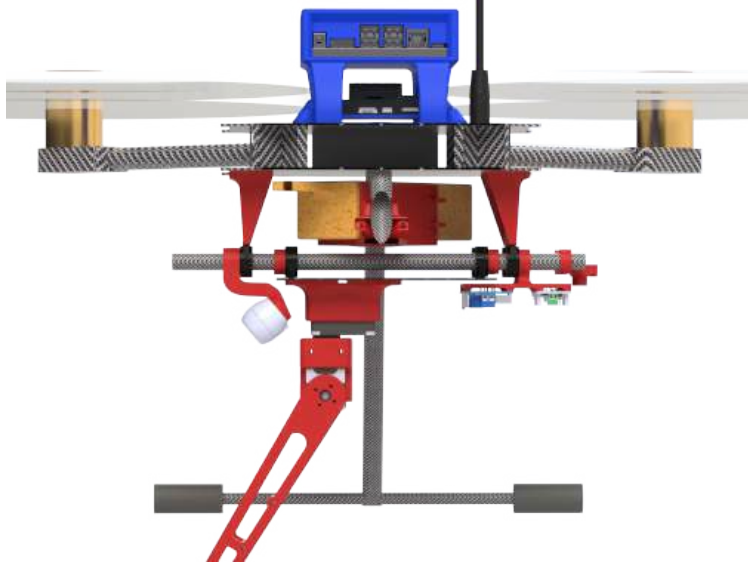


Figure 19: Vertical cutaway of the peripherals on the Terrawarden Aerial Manipulator

ROS’s communication relies on topics, which are identifiers for an open message channel. Each topic can only be used for one type of message, which is declared when the first publisher or subscriber on that topic is instantiated. ROS supplies a library of prebuilt message types, but also supports the creation of custom messages to transmit alternative data formats.

As a communication framework with specialized software libraries designed for robotic systems, ROS meets all the system architecture requirements for this project. Additionally, ROS’s nodal structure, where each node is an isolated process, allowed for isolated testing of each subsystem. This is especially true for both bench and unit testing, where running nodes individually is useful for preventing small issues which may be hard to find during integration. This project used the ROS2 Humble distribution, as it was the latest long-term-support release version of ROS available at the start of project development.

Node Structure

The communication network, shown in Figure 21, was managed by the central Task Manager node. This node monitored the drone’s current task and controlled the step-by-step processes and flow of information across the entire system. The other nodes act as a hardware abstraction layer, connecting the hardware elements to the command architecture. Although each node is an isolated process that controls portions of a subsystem, the overarching system was considered centralized because all decision making occurred in the Task Manager node. A simplified version of the node diagram, shown in Figure 22 visualizes this control process. Using a centralized system, governed by a single node, instead of a decentralized or distributed system simplified the testing of individual components of the project, as each subsystem was controlled by its own node. The difference between centralized, decentralized, and distributed node structures is shown in Figure 23. Although a fully decentralized approach was considered, where each node would simply pass information to the next without any sort of larger coordination, this approach would have gotten too complicated in the different modes of the system. A centralized approach using the Task Manager, in contrast, allowed for developing a state machine which directed how information should be interpreted and acted upon based on the operating state of the drone (see Section 3.6).

Although the Task Manager facilitated the primary communication between subsystems, some independent communication occurred as well. Hardware elements, such as the Pixhawk Autopilot, Dynamixel Actuators, and Intel RealSense, communicated with the external nodes, which functioned as a hardware abstraction layer, instead of directly with the Task Manager. These nodes processed the raw output from the hardware and relayed it back to the Task Manager, which used it to make an educated decision regarding the system’s next step. The Task Manager then relayed the relevant information to the external nodes, which



Figure 20: The Terrawarden Aerial Manipulator

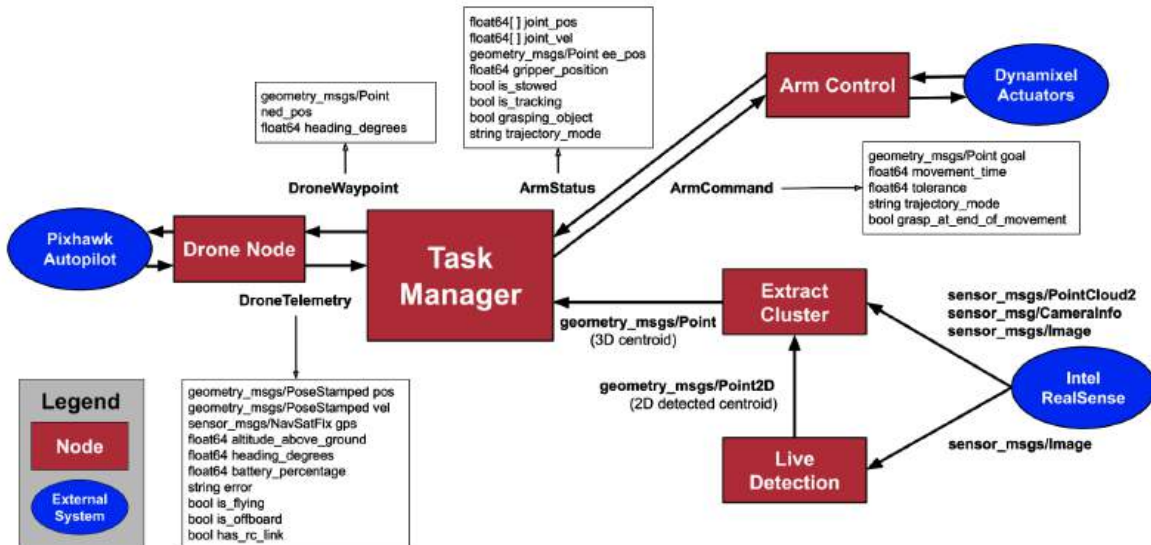


Figure 21: Full node diagram with message types and external systems

commanded the hardware to execute the process.

In order to effectively command hardware from the Task Manager, four custom message types were created: ArmStatus, ArmCommand, DroneTelemetry, and DroneWaypoint. Two of the message types, ArmStatus and DroneTelemetry, communicate status information about their respective systems, including the current position and velocities of the joints and end effector, and any relevant debug information or error flags. The other custom message types, ArmCommand and DroneWaypoint, are sent by the Task Manager to command the subsystems. They include target positions alongside any motion constraints. These commands are issued in the standard ROS2 local frame: front-left-up (FLU). The origin of this local coordinate system was placed at the center of the Pixhawk flight computer to align it with the drone frame.

Additionally, the overarching vision system, which consisted of the Extract Cluster node, Live Detection node, and input from the Intel RealSense, effectively functioned as a single independent process. The Task Manager could only process the 3D output from the Extract Cluster node, not the 2D output from the Live Detection node, so there was no need for a direct connection to the Live Detection node. Connecting the

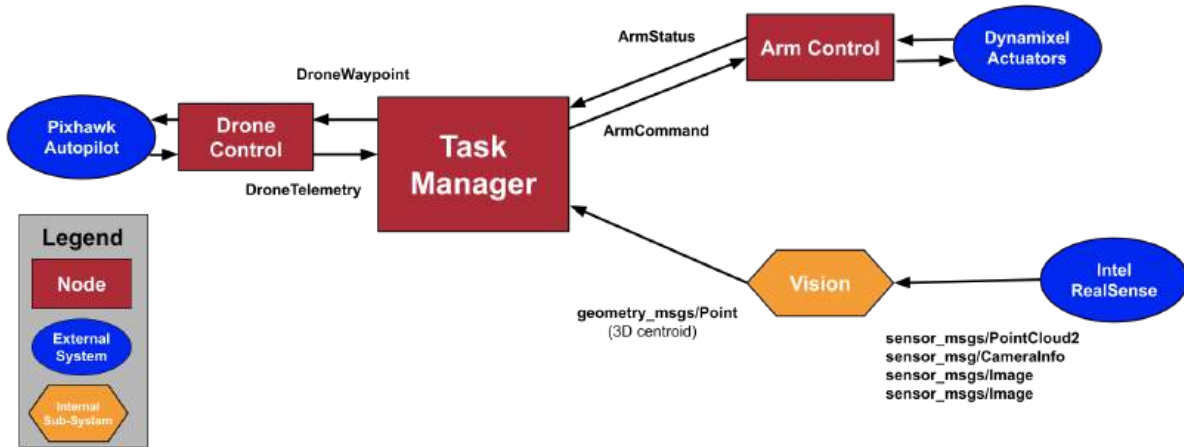


Figure 22: Simplified node diagram

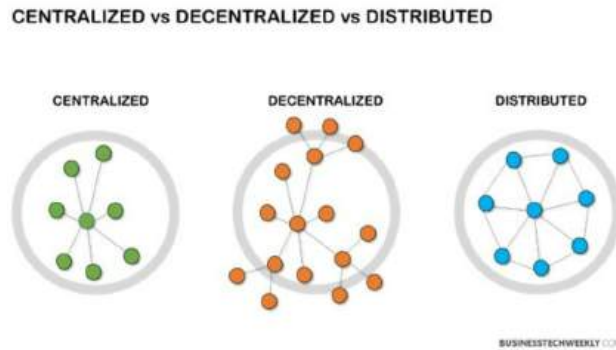


Figure 23: Difference between centralized, decentralized, and distributed node structures.

Live Detection node to the Extract Cluster node removed the Task Manager as a middleman in this process and simplified the code flow.

Overall, the centralized architecture allowed the external nodes to handle sensor processing simultaneously as data was sent to Task Manager for decision making. With this architecture, the Task Manager to continuously received up-to-date sensor info as it transitioned between states and decided what to do next.

3.3 Detection Model Training

One of the primary goals of this project was detecting trash on highway medians by feeding input from a camera into an object detection program. Modern approaches for object detection utilize deep learning, specifically convolutional neural networks (CNNs). As such, developing a CNN-based model capable of identifying soda cans from the perspective of a drone in flight was integral to the project's success.

Dataset Options

The amount of data needed to train a machine learning model increases rapidly with model complexity. To produce accurate results, the number of data samples a model requires is generally considered to be at least 10X the number of parameters it considers when training [11]. Object detection models train on images containing bounding box annotations, which are data points that frame objects and describe their dimensions, object type, and position within an image. Manually creating and annotating a collection of images large enough for a model to train on can be time-consuming or costly, so preexisting open-source datasets are often used in Computer Vision (CV) research.

	UAVVaste	TACO	Blender Generation	Project Need
Images Count	772	1500	Custom	Unknown
Annotation Count	3716	4784	Custom	Unknown
Number of Classes	1	60	Custom	1-3
Types of Litter	Mostly Bottles	Wide Variety	Custom	Cans, Bottles, and Bags
Perspective & Environments	View from a drone Fields & Paths	Ground View Varied Environments	Custom	View from a drone Highway Medians

Table 3: A comparison of dataset options vs project needs

Research was conducted to see if an open source dataset consisting of images of cans located between 0.25 and 6 meters from the camera existed. Two datasets, UAVVaste and TACO (Trash Annotations in Context), that matched the preliminary constraints were found. As described in Section 2.2, UAVVaste is a collection of 772 high resolution (3840x2160) images of litter taken from the perspective of a drone during low-altitude flight (between 3-5 meters). The dataset contains a total of 3716 annotations across the images, which are split into 12 categories, or classes [44]. TACO consists of 1500 images, almost twice as many as UAVVaste, but only has 4784 annotations since the pictures were taken from a much closer perspective. These images depict trash in nature, such as a plastic bag on a beach or a water bottle on the side of a road, and split the objects into 60 classes [63]. Example images from UAVVaste and TACO are shown in Figures 24 and 25.



Figure 24: UAVVaste sample image [44]

While UAVVaste and TACO were both viable dataset options, generating a custom dataset consisting solely of images of cans was also considered. Blender, a 3D-modeling and rendering program that can generate large quantities of training data using Python scripting, is often used to create synthetic datasets. While models trained on synthetic data require a larger dataset than those trained on real data, synthetic datasets can be easily expanded using PyTorch’s Torchvision transformation module. Each image can be augmented multiple times by adding transformations such as blur, noise, or color jitters (randomizing hues, brightness, saturation, and contrast), increasing the size of a dataset without the need to generate individual pictures. Additionally, the augmentation process, referred to as “Domain Randomization”, improves the performance of models trained on synthetic data or in a simulated training environment when they are transferred to real life [15].

Compared to TACO, the images in UAVVaste were a better match to the perspective, environment, and number of classes needed for this project. Due to the complexity and time required to build a custom dataset,



Figure 25: TACO sample image [63]

Blender generation was not initially considered and UAVVaste was chosen due to the overlap between its domain and the project needs. Table 3 provides a breakdown comparison of these datasets.

Iterative Process

Another important component of the detection subsystem was the model architecture. The initial model prototype used the MobileNetV1 architecture because it is a lightweight image processing model architecture compatible with the TensorRT framework, a Nvidia package used to optimize ML models for real-time processing on a GPU [88]. Open-source code for integrating ROS2, PyTorch, and TensorRT was found on GitHub and modified to create a training procedure, with a custom loss function which combined Generalized Intersection-Over-Union (GIoU) Loss and Cross-Entropy (CE) Loss [14, 68, 50]. Unfortunately, despite the model being initialized with random, non-zero weights, each prediction was a 0x0 sized box located at (0,0) with a confidence of 0, which signified that the model was not training properly. As the MobileNetV1 model had some crucial internal error preventing training, the model architecture was switched to match an open-source PyTorch architecture as it required less custom code and was therefore easier to debug [47]. This model was tested and was successful at identifying objects using pretrained weights from ImageNet however attempts to train it on the UAVVaste dataset were still unsuccessful. A closer examination of the UAVVaste dataset determined that in order to process the 4K resolution images from UAVVaste, the model scaled them down to just 224x224, making them too pixillated for the model to train. Modifying the model to process 640x480 images produced non-zero outputs, but the model accuracy was only about 5% post-training. As a >60% true positive rate and <10% false positive rate was needed for this project and improving accuracy on the UAVVaste dataset would be extremely challenging, the decision was made to use the TACO dataset instead.

However, after training the PyTorch model on TACO, the predictions still had an average accuracy of <10%. Since the model failed to train on two different datasets, the error was blamed on the training procedure and transfer learning, the concept of tuning a pretrained model for a new application, was attempted. Unfortunately, transfer learning did not improve the PyTorch model's performance, so a new architecture was selected. You Only Look Once (YOLO) is a highly documented, real-time object detection model maintained by Ultralytics that includes a Python library that simplifies the process of writing training code for their models [66, 39]. Ultralytics provides a way to visualize the training progress, which was used to track

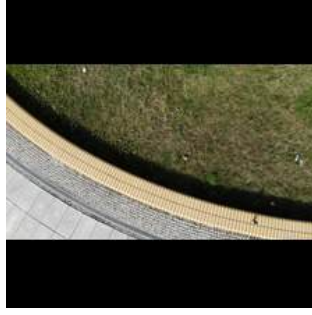


Figure 26: Sample from UAVVaste shrunk to 224x224. The original image is shown in Figure 24 [44]

the YOLOv11's performance when training on TACO. Figure 27 shows the training metrics used to measure performance of a model, including precision ($\frac{\text{\# of correct predictions}}{\text{total \# of predictions}}$) and recall ($\frac{\text{\# of objects detected}}{\text{total \# of objects present}}$)[80].

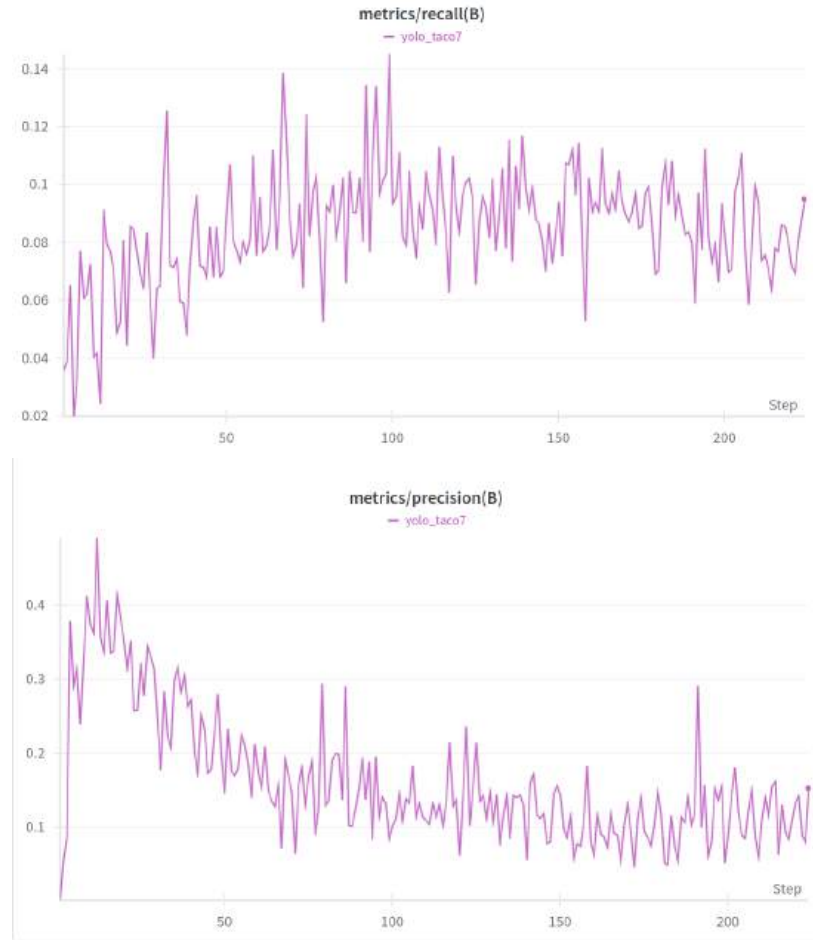


Figure 27: Attempts to train YOLO on TACO resulted in very noisy metrics

When training on TACO, the best iteration of the trained YOLO model only reached recall and precision accuracies of approximately 10%. As training had been unsuccessful on multiple architectures, these results were likely caused by a limitation of the TACO dataset. Although TACO contains 4784 annotations between 1500 images, the annotations are split across 60 classes, meaning there is on average only 80 annotations per class. As most models at minimum require thousands of data points to accurately detect objects, TACO

was no where near robust enough for the needs of this project.

To address this, the decision was made to generate a custom dataset using Blender. Three open-source soda can models were found online (see Appendix A) and used as the main training objects. These models were added to a Blender scene alongside primitive Blender models (such as spheres, cubes, cones, and tori) to partially obscure, or occlude, the can models. This process ensures the model is still able to detect cans that are slightly hidden and is using more than just the shape of the can to train. The items were placed at random locations and angles within the scene, creating a robust dataset that accounts for size and perspective. The images were initially generated with transparent backgrounds so that they can be added to a range of random backgrounds, making the dataset robust to environment as well. Bounding box annotations, data representing the desired model output, were added to the images using a Python script based on Blender's built-in matrix transforms. The base images were then color-jittered to limit the model's reliance on certain colors, saturation, and brightnesses. As rendering images in Blender is extremely time consuming and resource-intensive, only 5,000 images were generated, but these images were augmented using PyTorch transformations to expand the dataset to 50,000 images. This new dataset, named CaNET, contains over 65,000 annotations of cans and covers the range of inputs defined by the project constraints. The results of training YOLOv11 on CaNET can be seen in Figure 28. Compared to the training on TACO, this training has significantly less noise and reaches a recall of almost 90% and precision of over 95%.

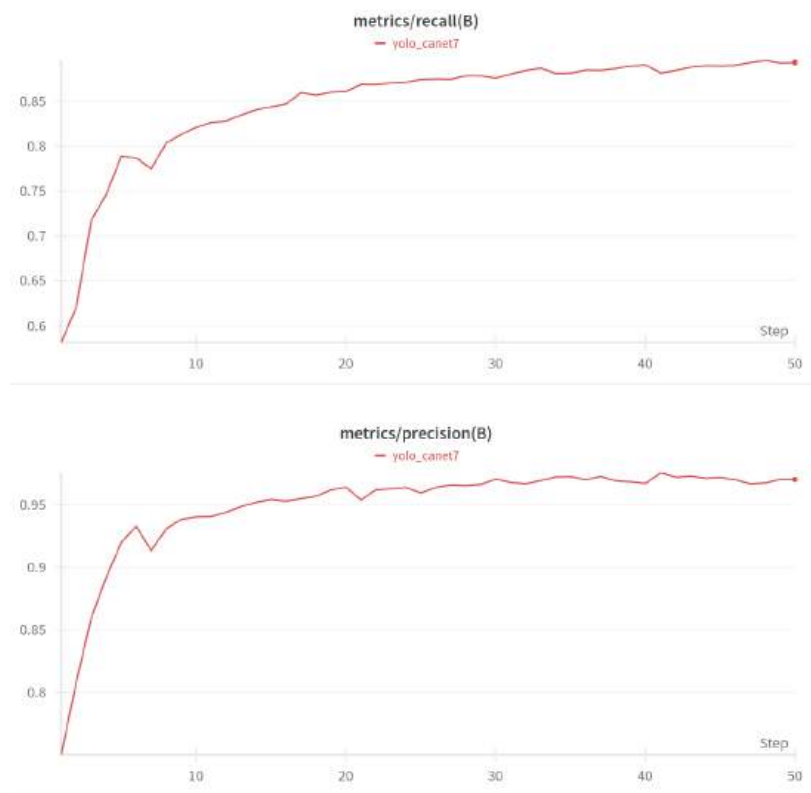
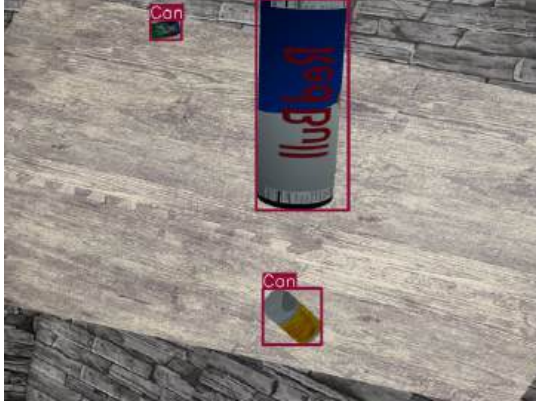


Figure 28: Training YOLO on CaNET was smooth and resulted solid metrics

Model Integration

While the YOLOv11 architecture was used in the final model training, the CaNET dataset went through a few additional iterations to address certain limitations. When tested in the real world, the model had an extremely high false positive rate because there were no non-can cylindrical objects (such as speakers and computer mice) in the dataset. To address this, additional backgrounds containing humans and objects in a laboratory setting were added to the dataset. Additionally, augmentation strength was reduced as many



(a) Color jittered can on background with bounding box annotations visualized.



(b) Generated image after a random series of blur, noise, and elastic transforms.

Figure 29: Visual comparison of a color-jittered image with annotations and a transformed version using blur, noise, and elastic effects.

images were becoming unrecognizable due to oversaturation, which was leading to over training and a high false positive rate.

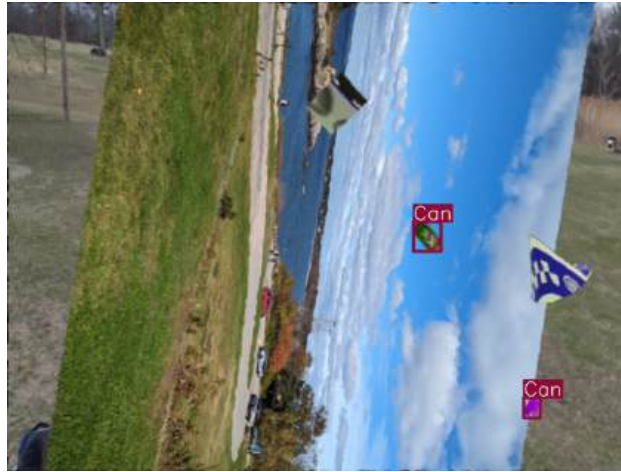


Figure 30: A final training example from CaNET v2

To integrate the model into the overall code structure, a simplified node, Color Detection, was initially designed. This node utilized OpenCV to create a color mask, and then identified the largest cluster of that chosen color. This was done for simplified integration testing, and the final node, named Live Detection, was integrated later, using the YOLOv11 model trained on CaNET. Both nodes fetch images by listening to the RealSense Camera node, which processes the image from the drone's Intel RealSense camera. The Live Detection node subscribes to and publishes messages of all the same types and on all of the same topics as Color Detection, making them interchangeable relative to the larger system architecture (as described in Section 3.2). These message types are:

1. Point2D, a data type containing the centroid of the most probable object detected within the image
2. Detection2D, which stores data for the largest detected item
3. Image, The model's visual output, consisting of the image with bounding boxes displayed around all the detected objects. This was used during testing for visualization and debugging.

This simplified node structure allowed for integration testing prior to the completion of the trained model. Additionally, once YOLO was done training on CaNET, it was seamlessly added to the image processing pipeline.

3.4 Manipulator Design

Manipulator-Arm

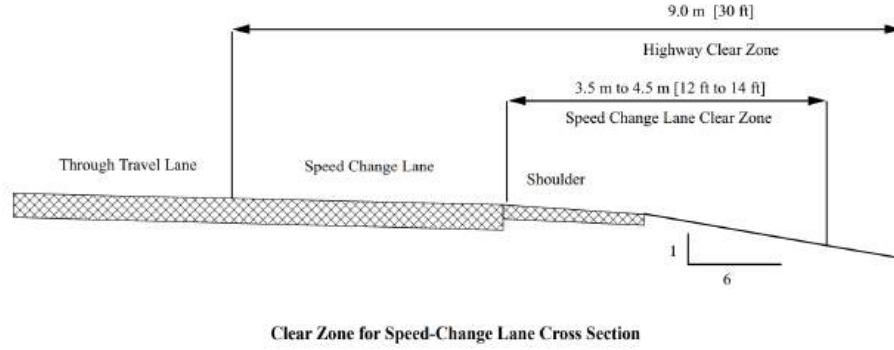


Figure 31: Highway median construction standards from the American Association of State Highway and Transportation Officials (AASHTO) [70]

To prevent cars from rolling into opposing traffic during an accident, highways medians are sloped downwards as shown in Figure 31. The slope of medians in the United States varies from a 1:6 vertical-to-horizontal ratio ($\sim 9.5^\circ$) to 1V:3H ($\sim 18.5^\circ$) depending on the width of the median and its purpose [70]. To prevent collisions between the drone's landing gear and the median, the manipulator design must account for the fact that litter is located on a slanted surface. Additionally, the manipulator's reach must extend past the propellers to prevent trash from being displaced by the drone's thrust. Based on these parameters, the minimum working values for the manipulator arm's vertical and horizontal reach were chosen as 28.4 cm below the carrying plate and 47.1 cm from the drone's COM as depicted in the calculations shown in Figure 32.

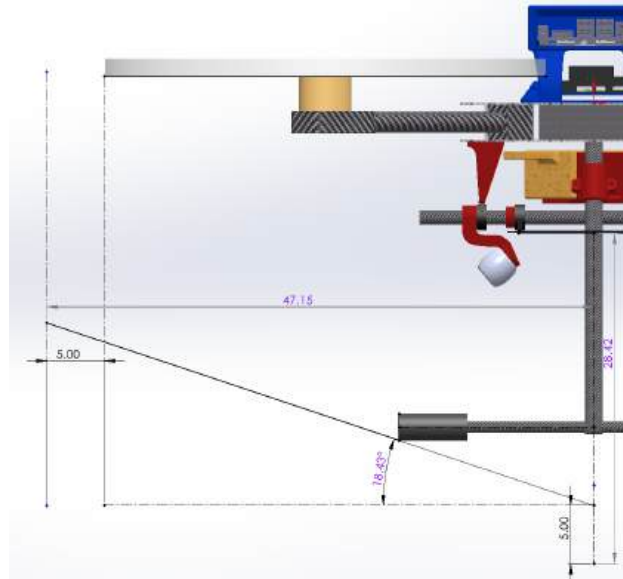


Figure 32: Diagram showing the minimum specification of manipulator reach before factors of safety (cm)

Additionally to balance the drone’s weight capacity of 1.5 kg between the manipulator, onboard computation, and payload, the manipulator system could not weight more than 0.5 kg or 1/3 of the max payload. To decrease system weight, a 3DOF manipulator design was chosen as this number of DOFs requires fewer actuators but still allows for full position control.

Mechanical Design The arm links, shown on the arm designs in Figures 35a and 35b, were designed in SolidWorks and are comprised of three pieces: two long, thin links that attach to the motors and a stabilizing plate connecting them. This design is lighter and easier to 3D print than a single structure but is still sturdy because it is stabilized by the connecting plate. The long connector links contain channels that interface with zip ties, creating a wire management system for the motor cabling. As the links have the same basic design, but different lengths, building the SolidWorks model using equation parameters allowed different sized links to be generated using a single model. Originally, the entire manipulator was 3D printed out of PLA filament as it was easily accessible and is a lightweight material. However, during testing, the arm links were consistently bending under the weight of the system. To fix this, these components were reprinted out of glass fiber reinforced ABS (ABS-GF) filament. Not only is this a stronger, more rigid material, it is also less dense than PLA ($1.08 \frac{g}{cm^3}$ compared to $1.24 \frac{g}{cm^3}$) which further decreased the manipulator’s weight. Reinforced filament is better than standard ABS for this application as the glass fibers increase the strength and rigidity of the material. Rigidity is desired because unpredictable flexibility can decrease the accuracy of the end-effector location, impacting the manipulator’s ability to precisely grasp objects. Additionally, the material’s composite properties decrease warping during 3D printing, increasing dimensional stability of the components. Another aspect considered in the manipulator design was the joint configuration, which determined the shape of the manipulator’s workspace. As there is essentially an infinite number of possible joint configurations, even the constraint of using three revolute joints meant that there were still three possible configurations: YYY, YXY, and ZYY. Joint configurations are named after the axis the rotate on in the world frame, shown in Figure 33.

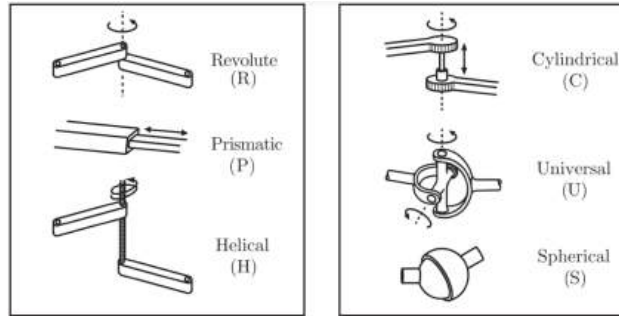


Figure 33: Examples of different types of robotic joints [49].

Each configuration had specific benefits and drawbacks, as shown in Table 4.

The YYY configuration was excluded because it could not provide the full positional control needed for reliable object grasping. However, as the YXY and ZYY both met the preliminary workspace design parameters, they were modeled and manufactured for further evaluation of their characteristics.

Kinematics The kinematics of the mechanisms were also considered when choosing the best joint configuration. Forward kinematics is the process of using the arm’s joint positions to determine the position of the end effector. The forward kinematics of each potential configuration were derived using the Denavit-Hartenberg (DH) convention, a popular approach for kinematically analyzing robot systems. The simplicity and flexibility of the DH convention provided a standardized method for representing the transformations between joints by using four parameters, α , a , d , and θ , which describe the change in position and orientation between each of the joints [24]. Inserting the parameter values for each joint into a transformation matrix creates a mathematical process for visualizing that joint’s location in the world-frame. This process is shown in Figure 37. Multiplying the transformation matrices for each set of DH parameters together creates an

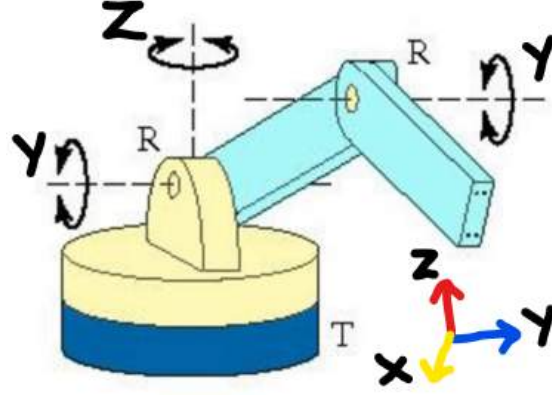


Figure 34: Example of a ZYY arm configuration [82]

Configuration Name	Advantages	Disadvantages
YYY	Allows for X, Z, and Pitch control	limited end effector position control
YXY	Minimal interference between drone landing gear and manipulator workspace Smaller workspace-to-manipulator-length ratio	Motor positions apply more torque on base motor
ZYY	Balanced torque requirements across motors Weight is concentrated at base instead of farther down manipulator Largest theoretical workspace	Workspace limited by drone legs

Table 4: Advantages and disadvantages of different joint configurations

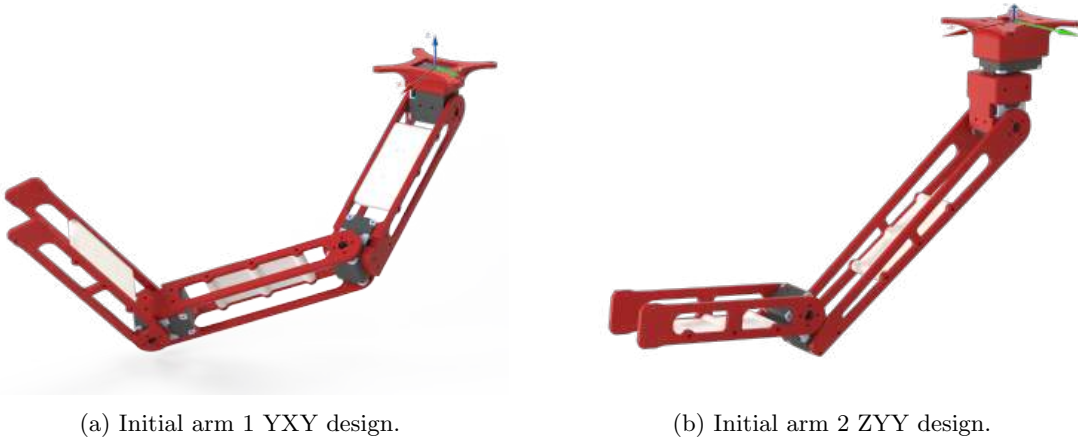


Figure 35: Initial arm designs.

equation that relates the end effector position in the world view to the joint position (θ values) of each joint.

Although the forward kinematics could be derived the same way for each configuration, the inverse kinematics required separate methods. Since the base joint of the ZYY arm is rotates in the Z plane and remaining joints rotate in the same plane, the bottom two joints can be modeled as a 2D planar arm on



(a) Workspace of YXY arm (all joints limited $(-\pi/2, \pi/2)$). (b) Workspace of ZYY arm (all joints limited $(-\pi/2, \pi/2)$).

Figure 36: Comparison of the workspace of the arms.

$$A_i = \text{Rot}_{z,\theta_i} \text{Trans}_{z,d_i} \text{Trans}_{x,a_i} \text{Rot}_{x,\alpha}$$

$$A_i = \begin{bmatrix} c_{\theta_i} & -s_{\theta_i} & 0 & 0 \\ s_{\theta_i} & c_{\theta_i} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & a_i \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & c_{\alpha} & -s_{\alpha} & 0 \\ 0 & s_{\alpha} & c_{\alpha} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_i = \begin{bmatrix} c_{\theta_i} & -s_{\theta_i}c_{\alpha_i} & s_{\theta_i}s_{\alpha_i} & a_i c_{\theta_i} \\ s_{\theta_i} & c_{\theta_i}c_{\alpha_i} & -c_{\theta_i}s_{\alpha_i} & a_i s_{\theta_i} \\ 0 & s_{\alpha_i} & c_{\alpha_i} & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Figure 37: DH transformation matrix generation

θ	d	a	α
0	-0.03825	0	$\frac{\pi}{2}$
$\theta_1 - \frac{\pi}{2}$	0	0.170	$-\frac{\pi}{2}$
θ_2	0	0.180	$\frac{\pi}{2}$
θ_3	0	0.180	$-\frac{\pi}{2}$

Table 5: DH Parameters for arm 1 YXY

the XZ plane (the vertical plane bisecting the bottom two joints). This allows the inverse kinematics to be solved geometrically using simple trigonometric equations. As the base joint of the YXY arm does not rotate in the Z plane, this strategy cannot be used and a numerical approach known as the Newton Raphson method, which finds an approximate goal from an initial guess using its derivative, must be used. A more in-depth breakdown of this approach is described in B. This method has longer, less predictable compute times compared to the ZYY configuration when the arm is used in flight. Due to its simpler kinematics, balanced torque requirements, and centralized weight distribution, the ZYY configuration was ultimately chosen for the arm prototype.

Arm Actuation Dynamixel XL430-W250-T actuators were chosen for the arm control because they fully integrate the functions of a DC Motor, PID controller, motor driver, encoder, gear reduction, and communication protocol in a single DC servo module. The Dynamixel actuators simplify the design and implementation of the arm and the integrated internal gearing eliminates the need for additional mechanical components, reducing complexity and points of failure. Dynamixel also provides an robust software library, Dynamixel SDK, used to communicate with and control their motors, this makes programming the motors extremely intuitive and allows for integration of the motor control protocols with the existing code base.

Unfortunately, when the arm function was tested with the gripper and the payload attached, the XL430-W250-T's did not generate enough force to lift the arm for a sustained period of time and would disengage

θ	d	a	α
$-\theta_1$	-0.03825	0	$\frac{\pi}{2}$
$\theta_2 - \frac{\pi}{2} + 0.0715$	0	0.2381	0
$\theta_3 + \frac{\pi}{2} - 0.0715$	-0.01	0.25	$-\frac{\pi}{2}$

Table 6: DH Parameters for arm 2 ZYY

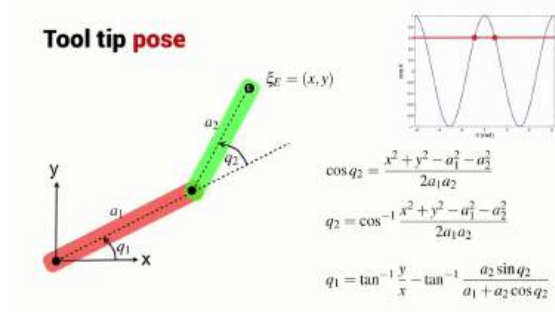


Figure 38: When a 3 DOF manipulator has two consecutive joints in the same plane, that portion of the arm can be modeled as a 2D planar arm. This allows the inverse kinematics for those joints to be solved geometrically using the equations for q_1 and q_2 above. The base joint angle, q_0 , is equal to the angle between the X axis and the end-effector's (x, y) coordinates, $q_0 = \tan^{-1}(\frac{y}{x})$ [19]

to reduce strain on the gears. As Dynamixel offers both plastic cored (XL) and metal cored (XM) motors, replacing the current motors with the metal cored XM430-W350-Ts improved the strength of the motors without needing to redesign any of the components. For a small, additional torque increase, a XM motor with a higher internal gear ratio (350:1 compared to 250:1) was chosen. Overall the switch to a metal cored motor with a larger gear ratio increased the stall torque from 1.4 Nm to 4.1 Nm. An additional benefit of this switch was an increase in motor precision, which improved the arm's ability to precisely align with target point and increased the grasp success rate.

Arm Control Arm control occurred in the Arm Control node, which listens for arm commands, (which include goal positions, movement times, trajectory types and gripper movements) from the Task Manager node. When the Arm Control node receives a goal end effector position, it generates a quintic trajectory (fitting a parameter to quintic polynomial factoring position, velocity, and acceleration) in either the joint (control over individual joint angles) or task (control over end effector XYZ position) space and uses it to navigate to the object. While the arm is in motion, the Dynamixel's integrated controller uses position goal commands to update the difference between the desired trajectory and the actual motion at a rate of 100 Hz. After each update, the controller attempts to adjust the motor's position to match the trajectory. Because the Arm Control node directs each motor to the point it ideally should already be, the arm motion slightly lags behind the simulated trajectory times. This occurred during all arm motion except grasping, where the tracking movement bypass the trajectories and are sent directly to the motor's onboard velocity based interpolated profile. This prevents lag from interfering with time-sensitive tasks and ensures the trajectory is not reset if a grasp position is updated. Every 10 ms, the Arm Control node sends a status message monitoring the current joint positions, joint velocities, end effector position, gripper position (open/close), and whether there is an object within the gripper to the Task Manager. By monitoring this information, the Task Manager can automatically stop any processes that appear to be functioning incorrectly.

Manipulator-Gripper

When designing a gripping mechanism for an aerial manipulator, there are several constraints that must be considered. Since the drone could only support a manipulator assembly with a maximum weight of 0.5 kg, the mechanism needed to weigh less than 0.2 kg. The gripper had a payload of 0.4 kg, approximately the

weight of a standard 350 mL can of soda, and needed to be able to grasp it in flight without the object slipping or dropping. Additionally, to account for variability in the drone’s position hold, the gripper needed to be versatile, able to grasp objects from various orientations and positions. Versatile grippers also improve vision-based manipulation as their robustness compensates for any inaccuracy in the vision pipeline.

Initial Prototype Gripping mechanisms and finger design can largely be classified into two categories, hard robotics and soft robotics. Hard robotics, the more common category, involves mechanisms made from rigid materials, such as metal, PLA, or ABS, while soft robotics describes the subset of mechanisms made of pliable materials like rubber or TPU [3]. Soft robotic mechanisms are generally more versatile than hard robotic systems, especially for simpler, limited DOF mechanisms, but their flexibility means they cannot be evaluated using standard rigid-body force analysis methods. As soft plastic components are more challenging to manufacture and have more points of failure, the initial finger design for this project was a rigid, hard robotics mechanism.

This design, shown in Figure 39, is a rigid mechanism that was designed purely for grasping cans. Each finger had a curved section with a radius of 33 millimeters, the radius of a standard soda can, and was hooked inwards at the end to put pressure on the can and prevent it from slipping out of the grasp. The entire mechanism, shown in Figure 39, is composed of three fingers, two connected directly to the actuator and one controlled through a shaft-gear mechanism.

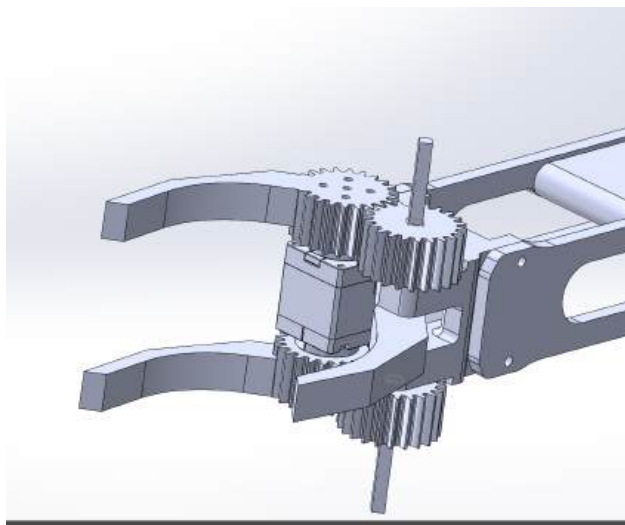


Figure 39: Original gripper design

Although this design interfaced well with the manipulator arm and was easy to control, there were a few major problems with it. Overall, the mechanism was simply too small, which prevented it from grasping a can in the desired way. The curved sections both had an arc length of 90 degrees and were supposed to interface with opposite sides of the can. However, the gear mechanism pushed the can too far forward, which meant the only part of the mechanism the can had contact with was the fingertips. The lack of surface contact also caused the cans to slip inwards and interfere with the gears. Additionally, PLA has a relatively low coefficient of friction, which meant that even if a can was grasped securely with multiple points of contact, it would still slip over time. During testing, an empty can (~ 18 g) slipped out within 60 seconds and the gripper was completely incapable of holding a full can (~ 392 g). Due to the flaws in this design, more research was conducted, focusing on existing gripper designs for trash collection. Although some studies had been conducted using hard robotic grippers, the majority of existing prototypes utilized soft robotics [4, 5, 17, 26, 34, 43, 73, 81]. Of the soft robotics grippers in current literature, most utilized a phenomenon known as the Fin Ray effect to create a strong, adaptive grasp[17, 25, 52, 73, 87]. As slippage and limited versatility were problems that would be hard to address on the initial design, even if the finger design was adjusted, the decision was made to switch to a soft robotics gripper based on the Fin Ray effect.

The Fin Ray Effect The Fin Ray effect, discovered by Lief Kniese in 1997, is a soft robotics concept that mimics the movement of a fish’s dorsal fin. When a force is applied to it, the fin deforms towards that force instead of away from it, “catching” it in the fin [23]. This effect is shown in Figure 40. This happens because the fin has two rigid bony structures connected by elastic tissue. When the fin is pressed, the bones are pressed outwards, and the elastic tissue curves due to stress. Kniese discovered that by creating a structure with rigid crossbeams and an elastic frame, the Fin Ray effect could be replicated in robotic manipulators [23].

Fin Ray grippers conform to the object that they are holding and are useful for delicate processes because the force they use to grasp an object is proportional to the normal force the object exerts on the gripper [75]. While most roadside trash is not fragile, a Fin Ray gripper’s delicate grasp prevents items like Styrofoam cups or plastic water bottles from crushed and dropped due to loss of contact between the object and the gripper’s surface during flight.

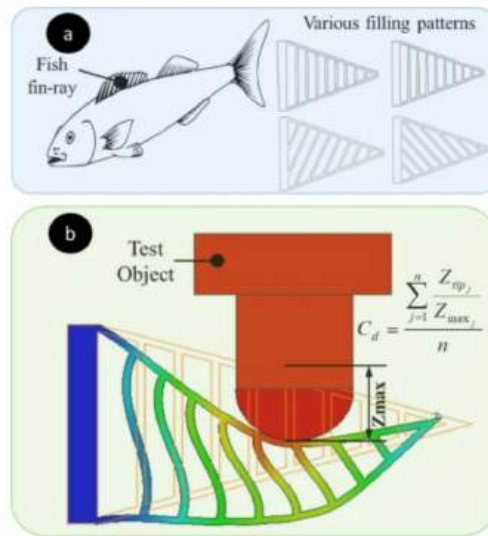


Figure 40: Depiction of the Fin Ray Effect [75]

Despite the advantages of using the Fin Ray effect, it is also difficult to maximize and hard to implement. The flexibility and strength of a Fin Ray gripper is governed by three main parameters: the crossbeam angle(s), the ratio between the height of the gripper and the number of crossbeams, and the material type [17, 25, 73]. When designing Fin Ray grippers, researchers generally use Finite-Element Analysis (FEA) to find the optimal values for these parameters [17]. Unfortunately SolidWorks lacks support for flexible plastic and assumes all bodies are solid, which meant the prototype’s real-world behavior was not accurately represented by the simulation. Because of this, designing a Fin Ray gripper without an external reference was not feasible within the project timeline. Fortunately, on December 17, 2024, Cong et. al released a study analyzing a reconfigurable Fin Ray gripper with tactile skin (the TacFR-Gripper). In addition to being a helpful resource for understanding the Fin Ray effect, the project website also included open-source engineering drawings of the TacFR-Gripper design[17]. While the mechanism, shown in Figure 41, was too complex and heavy for this project, the finger design could be adapted to interface with a simple, lighter mechanism that better matched the project needs.

3D printing flexible plastics is extremely challenging because their high elasticity causes the material to sag and warp while it dries. For this project, access to 3D printers was limited to plastics with a Shore hardness of 83A or higher. Since the TacFR-Gripper was originally printed out of 95A TPU, this constraint did not limit the manufacturing feasibility of this design. A more in-depth depiction of the Shore hardness scale is shown in Figure 42.

The TacFR-Gripper finger design met the project requirements as Cong et. al already proved it could grasp a soda can in a two finger configuration [17]. The original engineering drawing taken from and the interpolated CAD model are shown in Figures 43 and 44.

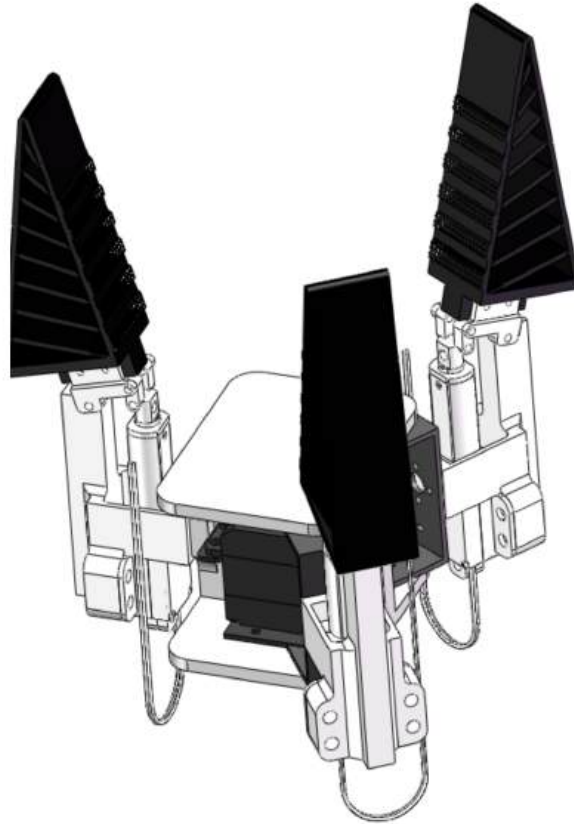


Figure 41: Fin-Ray Gripper Design from Cong et. al [17]

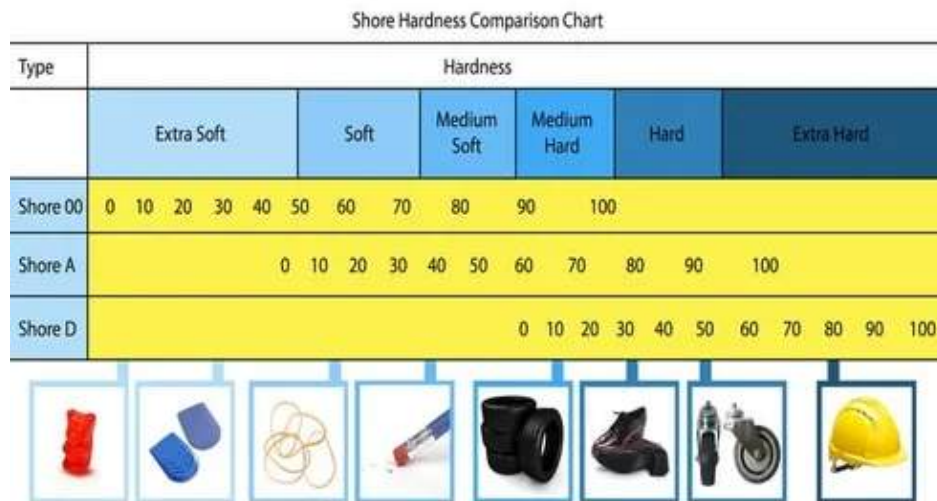


Figure 42: Visualization of the Shore Hardness Scale [83]

Gripper Actuation The actuator powering the gripping mechanism is a Dynamixel XC 330. This actuator was chosen due to its higher stall torque compared to similar models, improved durability with its metal bearings rather than plastic, and because it was small and light. Despite the finger design being finalized, multiple prototypes were still needed to create a mechanism capable of actuating both fingers with a single

moved fluidly in simulation, simulation did not account for the fact that there were too many degrees of freedom and the fingers' movement was not symmetrical, like it needed to be to achieve the maximal Fin Ray effect and therefore a strong grasp.

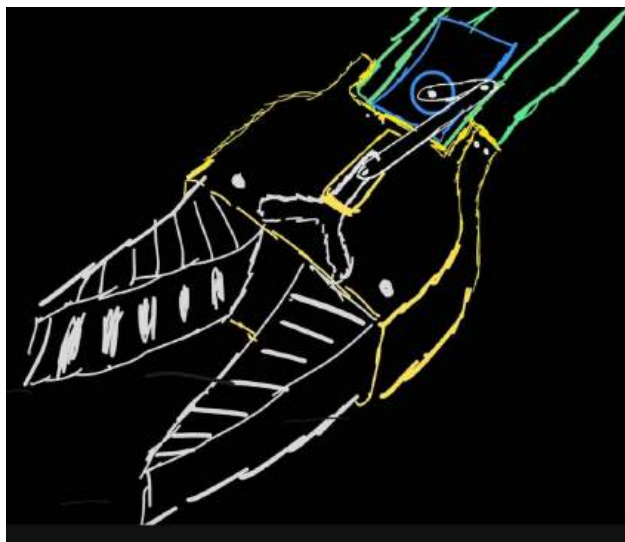


Figure 45: Initial sketch of the Fin Ray housing design with Y-shaped connector

The final design involved two gears, one which is directly connected to the motor and one that is screwed into the gripper body. Each gear is connected to the gripper finger via a link. Although the mechanism moves as one, the rigidity and mirrored motion of the gears allows the system to function as two separate four-bars, eliminating any extra DOF. Lastly, the gear-and-motor assembly was attached to the gripper body and mounted on the manipulator arm. The gripper body underwent multiple design iterations as well, focusing on reducing weight and friction, improving strength, and preventing gear slippage. This final assembly can be seen in Figures 46 and 47. All gripper components apart from the fingers were printed using ABS, chosen for its strength and lightness.

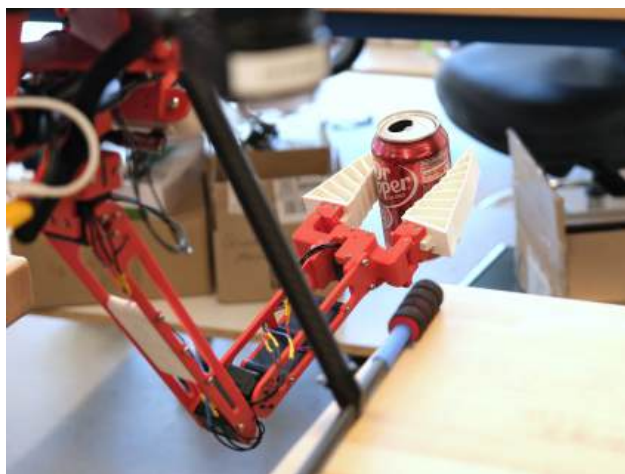


Figure 46: Manipulator grasping a Dr. Pepper Can

3.5 Vision-Based Manipulation (VBM)

An additional component of aerial manipulator control is the system's ability to perceive and respond to its environment. For this project, once the Live Detection node located a can, the system needed to a way



Figure 47: Curving of the Fin Ray fingers when grasping a cylindrical object



Figure 48: Intel RealSense Depth Camera D455 [20]

to locate where that object was in the world frame. Vision-based manipulation (VBM) uses visual data to control the manipulator, letting the system detect trash, localize itself, and grasp objects in a dynamic environment. Compared to traditional pre-programmed or other sensor based approaches, VBM is adaptable and more robust in dynamic environments [74].

Conventional two-dimensional cameras are generally not robust enough to determine the distance to an object, the object’s size, and detailed information about potential grasp contact points (see Section 2.4). Fortunately, stereo depth vision cameras, such as the one shown in Figure 48, address this problem by drawing inspiration from human binocular vision. By combining the output of two fixed cameras separated by a known distance and calculating the difference between a point’s position in each camera’s field of view, the system can estimate the depth of any given point. This process is illustrated in Figure 49 [27]. When this process is applied to every point detected by both cameras, the image can be rendered as a point cloud, a 3D representation of the environment’s features and any object surfaces. These point clouds can be processed and filtered to enhance, remove, or spotlight certain environmental features [72].

As the project required a stereo camera capable of quickly processing visual data in flight, the Intel RealSense D455 was selected for its fast, high-resolution 2D and 3D imaging as well as its compatibility with ROS2. Integrating 2D RGB data from the Live Detection node (see Section 3.3) with the 3D point cloud generated by the RealSense provides the system with data that can be used to localize targets, plan optimal grasps, and execute motions for the manipulator arm and gripper.

Once a can is identified by the Live Detection node, a 2D representation of that object is sent to the

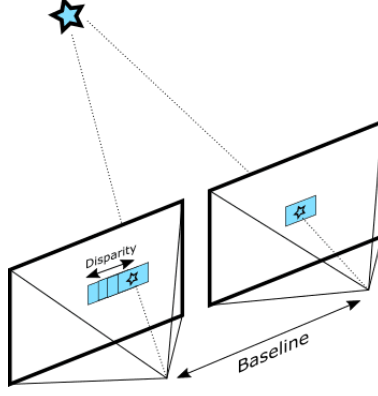


Figure 49: A stereo camera captures the same point in space through each lens. The disparity between these points allows for calculating the 3D distance to the point [27]

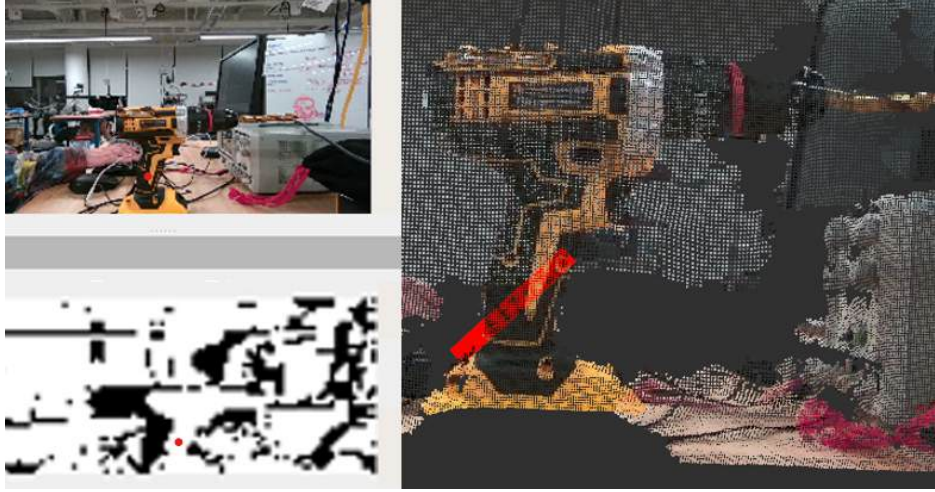


Figure 50: The RealSense publishes a color image (top left), a depth image (bottom left), and a point cloud (right). The program converts a given 2D point in the color image to the depth image and then to the point cloud, as shown with the red dot / line.

VBM Extract Cluster program, which converts it into a 3D point cluster in space. It converts the object's centroid from a 2D point to a 3D point by mapping the point from the 2D color image to the 3D depth image, then integrates it into the point cloud as shown in Figure 50. This process requires scaling the pixel coordinates using the resolutions of the color and depth images using Formulas 6 and 7, as each of the color and depth images is sent at a different resolution by the RealSense to prevent overloading the Jetson.

$$u = \text{centroid}_x \times \frac{\text{image_width_depth}}{\text{image_width_color}} \quad (6)$$

$$v = \text{centroid}_y \times \frac{\text{image_height_depth}}{\text{image_height_color}} \quad (7)$$

Next, the Extract Cluster node attempts to use the scaled 2D position computed above (u, v) to retrieve the object's depth in meters from the 3D image using Formula 8. However, the RealSense does not generate the depth map when a camera is occluded or the object is in a position that exceeds the minimum or maximum depth limits of the sensor, so this method is not always successful.

$$z = \text{depth_image}(v, u) \times 0.001 \quad (8)$$

Using the pinhole camera model, as demonstrated in Figure 51, the 3D coordinate is converted from (u, v) coordinates in the depth image to (x, y) coordinates in the camera frame. The 3D coordinates are calculated using formulas 9 and 10, where c_x and c_y represent the principal point coordinates of the camera and f_x and f_y represent the focal lengths in the x and y directions.

$$x = \frac{(u - c_x) \cdot z}{f_x} \quad (9)$$

$$y = \frac{(v - c_y) \cdot z}{f_y} \quad (10)$$

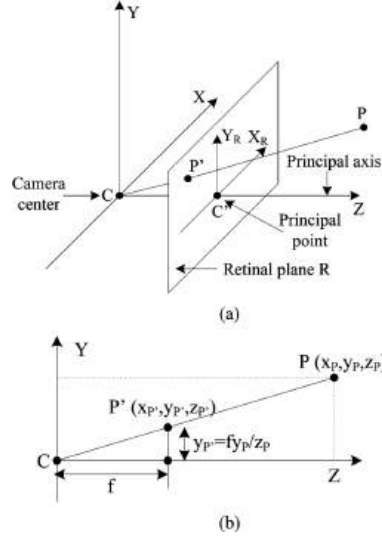


Figure 51: Geometry of a pinhole camera model used to convert a point from 2D into 3D space [85]

Finally, the resulting 3D point (x, y, z) is transformed from the camera frame into the drone's coordinate frame using ROS2's transform lookups, which aligns the point within the drone's reference frame before it is processed any further. These conversions are visualized using Gazebo physics simulator in Figure 52, which allowed the program to be tested with ROS2 in simulation prior to system integration.

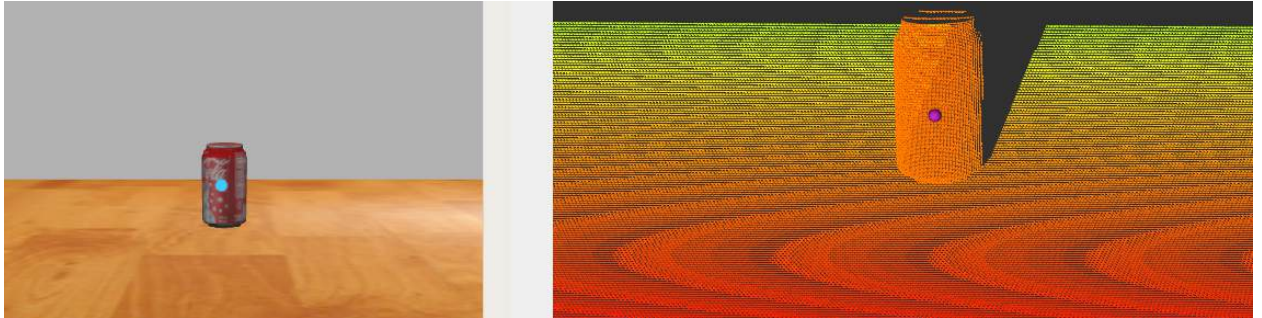


Figure 52: Using Gazebo simulator, a simulated depth camera publishes a 2D image (left) and a 3D point cloud (right) to RViz. A 2D point representing the centroid of the detected object (light blue, left) is converted to a 3D point (purple, right).

Once the centroid of an object has been localized in the drone's coordinate frame, the Extract Cluster program segments the point cloud around this point to obtain a point cluster representing the entire object. The node utilized the Point Cloud Library (PCL), an open-source library for processing point clouds with

native ROS2 integration, to filter out unnecessary points based on a series of user-specified parameters [72]. This improved computation time, which meant the process could be completed in real-time. The node then segments the point cloud around an object by cropping it to a small region around the 3D point, removing noisy measurements using a statistical outlier filter. A 3D voxel grid filter reduces the number of points even further, by decreasing the point cloud resolution. Finally a plane segmentation filter is applied to separate the points representing the object from points representing the ground. To ensure this point cluster accurately corresponds to the desired object centroid, PCL Euclidean clustering segments objects from the filtered point cloud and checks if they contain the original 3D point (within a specified margin of error). If the original 3D point is present in the point cloud, the program concludes that the cluster accurately represents the target object detected by the 2D camera. The program then uses the point cloud cluster to check the dimensions of the object, and returns a warning if the object appears to large for the gripper to grasp. If the object is an acceptable size, the point cluster is sent from the Extract Cluster program to the Optimal Grasp program for further processing. Some of these filters can be visualized in Figure 53.

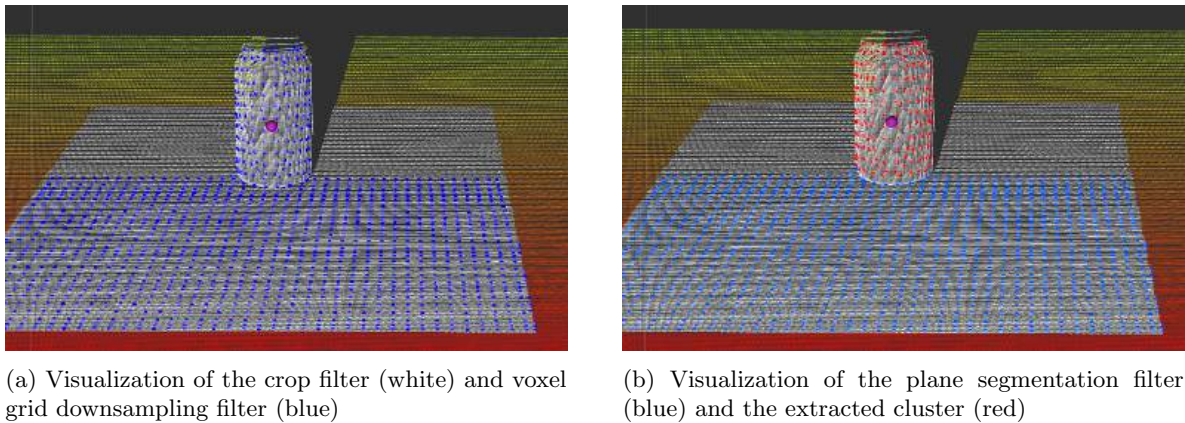


Figure 53: Visualization of the PCL filters applied to extract the soda can in ROS2 RViz

The Optimal Grasp program uses PCL to calculate surface normals on the point cluster of the trash candidate (see Section 2.4). The program estimates the relative curvature of the surface normals and eliminates any under a certain threshold, filtering for stable points that a gripper could grasp. It then searches through combinations of the remaining points and selects the pair of points with the highest grasp quality. To do so, the program calculates the algebraic properties of the grasp matrix formed by the two points and selects which contact points the gripper could apply force to and optimally grasp the object (see Section 2.4). If the points are too close or too far away based on the workspace of the gripper, the program will not consider them. The surface normals and resulting grasp can be viewed in a simulated environment in Figure 54. The drone and manipulator use the information from this program to navigate to the object and plan a trajectory for the manipulator.

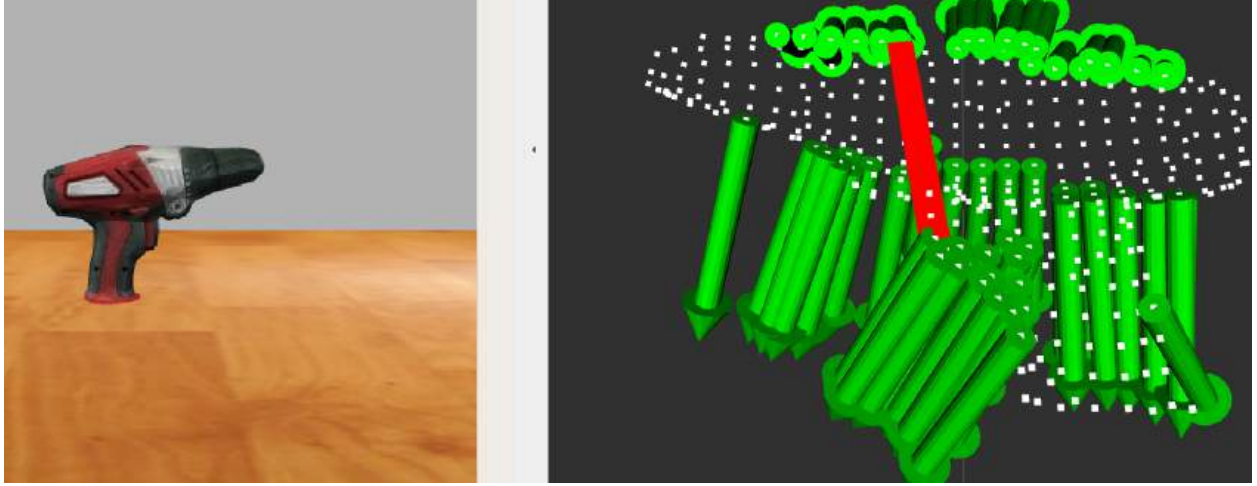


Figure 54: Gazebo simulator (left) and ROS2 RViz (right) were used to test the VBM pipeline. The stereo camera in Gazebo takes 2D and 3D images of the soda can, along with part of the table. The final processing done by the VBM programs are shown in RViz - the green arrows represent stable surface normals from the filtered cluster, and the red line shows the calculated grasp.

3.6 System Integration

State Machines are a common software design pattern, where different behaviors are defined with strict transition conditions. By having multiple states, non-blocking code can be conceived such that the system can perform error-checking and data-logging isolated from the step the code is on. Each state is also able to be tested in isolation, allowing for easy debugging of individual behaviors. The flexibility of the state machine pattern is also very useful for adding new behavior, as occurred multiple times through the development process [76].

Within the Task Manager node, there are nine states, which follow an automated sequence flowing one into another, or can be manually set by the operator through the `set_state` command. Not pictured in Figure 55 is the Fail-safe or Landing states, which are an emergency sub-procedure that can be entered from any other state to try and recover the drone if something goes wrong, such as the RC link being lost.

Startup State is designed to initialize the ROS nodes and monitor incoming inputs, preparing the system for a transition to autonomous control. During this phase, the drone remains under manual control while the autonomous system passively observes, performing safety checks to ensure the system is ready for autonomous operation.

Hold State is intended to function as a safety buffer while initializing a new state. In this state, the drone and manipulator position are held static. This state is the default drone state if any error were to occur. Only upon receiving a `set_state` command does the drone change state, to prevent a critical failure post-error and to ensure the drone is stable before transitioning into Search.

Search State commands the drone fly along a predetermined search pattern. For the purpose of indoor testing the pattern is reduced to a slow clockwise spin in place. The search pattern repeats until a viable target object is detected by the vision system, after which the drone waits a few seconds, which allows for an operator to cancel the procedure if a false-positive or error occurs, then switches into Navigate.

Navigate State is where the drone approaches the target object, until the object is within the manipulator's workspace. This is intended to get closer to the object, and to align the drone's heading so that the object is centered in the camera frame. Once this occurs, the drone transitions into the Grasp state.

Grasp State generates and executes an optimal grasp - the manipulator moves towards the object's location

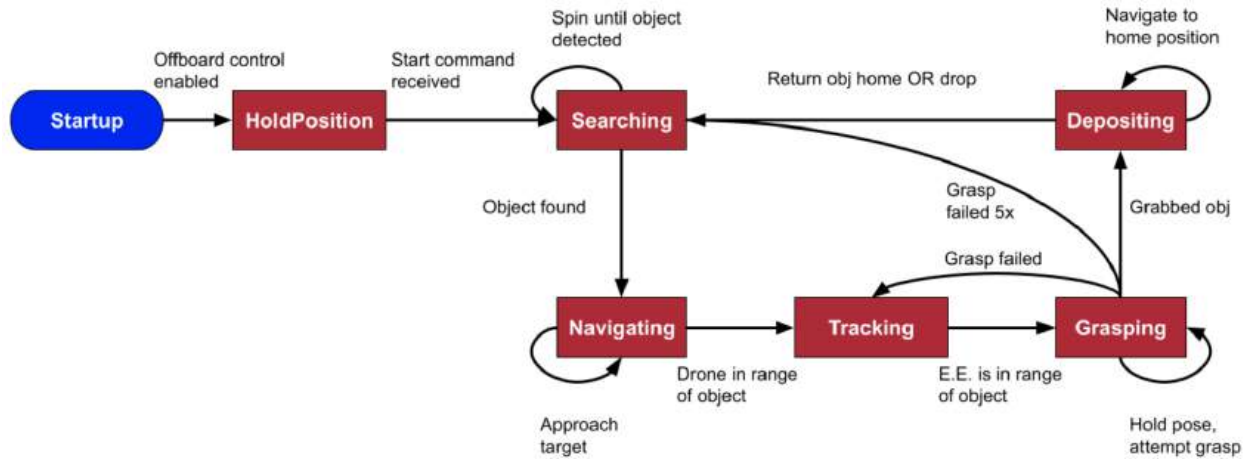


Figure 55: The finite state machine within Task Manager

while the drone hovers in its position. Once the end-effector is confirmed to be at the object, the gripper's jaws close to grasp it.

Deposit State Once the object has been grasped, the Deposit state guides the drone to its home position to deposit the litter. In the current implementation, 'deposit' just refers to dropping the object from above the home position, but in the future could refer to autonomously detecting a recycling bin or trash can for disposal. Afterwards, the drone returns to the Search state.

Wait State The Wait state is very similar to Hold in function, with the key difference that it allows the drone and manipulator to keep moving, so that all processes running can finish. This buffer is designed to wait either a certain amount of time before transitioning to the next state, or for a boolean condition to be true. An example of this is the transition between Search and Navigate, where the manipulator continues unstowing itself before finishing the state transition.

Fail-safe State The Fail-safe state was created to cover a potential failure state in which the remote control connection is lost and the drone being autonomously controlled by the onboard computer. In this state, the drone will return to home, flying above the take-off position, or until the remote control link is restored. In the event of an onboard computer failure, the autopilot is configured to automatically perform a similar return to home procedure. In addition, if the battery level is too low, the system will also enter the Fail-safe state.

Land State The Land state is a terminal state after the Fail-safe state, for when the drone returns above its starting position. In this state, the drone begins a slow descent while remaining in autonomous mode until it touches down on the ground, at which point it terminates the flight.

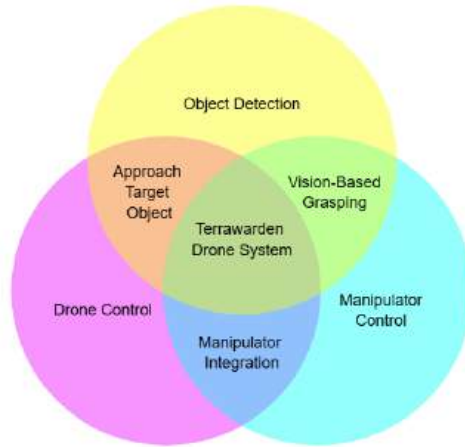


Figure 56: Graphic representation of the integration steps taken

These states form the high-level system control loop. In a fully autonomous scenario, the system is deployed and, following take-off, cycles through these states to perform cleaning tasks. Once the battery is depleted, the system initiates a return-to-home sequence and lands safely.

Using Task Manager and its state machine, subsystems were integrated one at a time, first with the vision stack, then the manipulator, then the drone. Once each subsystem was capable of communicating with Task Manager, two-subsystem tests were performed. Only once each subsystem was integrated with each other in testing did the full integration of all three sections begin. A representation of the integration process overlap between the nodes can be seen in Figure 56.

4 Results and Experimental Evaluation

A series of experiments were conducted on the drone and its individual subsystems to evaluate performance across key areas, including tracking the drone’s position, vision-based object detection, and manipulator actuation. The experimental setups included primarily indoor tests designed to assess the robustness and adaptability of each component, as well as their integration. Both quantitative and qualitative results demonstrated accurate drone localization, reliable object detection and tracking, effective manipulator payload handling, and seamless coordination between perception, actuation, and navigation. The complete system’s performance on the initial objective, detecting, navigating to, grasping, then depositing a partially-filled 12 oz can, was subsequently evaluated with promising results. The prototype’s performance supports the feasibility of using autonomous aerial manipulators for trash collection on highway medians, but also exemplifies why both subsystem-level testing and overall system integration are integral to success.

4.1 Drone Performance

Position Tracking

Testing the drone’s ability to hold its position began with one-minute autonomous hover tests, conducted both while the manipulator arm was in its stowed and unstowed (shown in Figures 57 and 58). These tests were performed to determine if the manipulator’s weight and location impacted the drone’s position tracking.

During these trials, the power output of each propeller motor (measured as a percentage of the max throttle) and the drone’s horizontal and vertical drift were monitored. Previous testing during development had shown that when the manipulator was not connected to the drone, all four motors operated at roughly 50% throttle and the drone held position without any visible drift. These values were used as a baseline to quantify how the manipulator impacted the drone’s performance. Compared to this baseline, when the manipulator was in the stowed position the throttle from the front motors climbed to about 75% - approaching the safety margin of 80% - while the rear pair remained near 50%. When the system was tested in the unstowed position, the motor throttles did not change significantly from the stowed values. These observations verified that the drone could perform aerial manipulation without exceeding its thrust capacity.

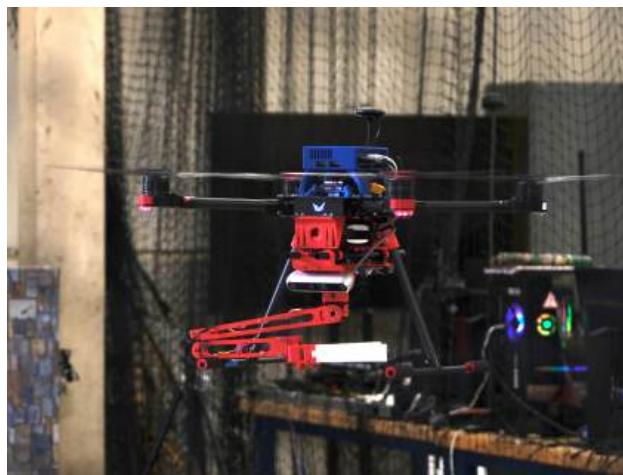


Figure 57: Drone holding position with the manipulator stowed.

After confirming that the flight control system could handle the load of the manipulator in any position, more in-depth position tracking testing was done. In addition to more thorough testing of the drone’s ability to hold position with the manipulator stowed and unstowed, its performance while the manipulator was actively moving in mid-air was also analyzed. Each test was initially carried out in manual control mode to establish baseline familiarity, then switched to offboard control to validate the autonomous position-holding routines. No external motion-capture system (e.g., Vicon) was employed – the goal was to validate performance in a realistic setting. Video analysis revealed very little drift and oscillations (which occurred



Figure 58: Hovering drone with the manipulator in the unstowed position, ready for grasp.

2-3 times every 10 seconds) were quickly dampened and had minimal impact on position tracking. The occasional drift and slight oscillations were attributed to accumulation of sensor uncertainty over power-on time and ground-effect aerodynamic disturbances, as these tests were performed indoors 1 meter above a flat floor. Sensor uncertainty was hypothesized to be the cause of the rare cases of position hold failure, mainly characterized by a gradual upward bias, as well because they mostly occurred after about 30 minutes of cumulative operation, especially following multiple takeoffs, landings, or controlled crash landings. Since this form of position tracking error was easily fixed by rebooting the flight computer, which reset the onboard sensor-noise estimators, a preventative solution was not explored.

It was also necessary to evaluate the drone’s yaw control as it needed to be capable of rotating in place to align with the target heading. If there was too much error in the yaw position, the can would end up out of the camera’s frame of view during the approach and the trial would fail. During initial trials, the drone lost altitude if rotation commands were too aggressive because the load of the stowed manipulator caused one motor to approach its maximum continuous-thrust limit when rotating clockwise. This was fixed by reducing the yaw-rate parameters and the drone was able to perform smooth, controlled rotations with minimal altitude loss in subsequent trials.

The system’s ability to track a trajectory to a distant setpoint under position control was also evaluated. At first, the drone surged forward and experienced massive dips in altitude when approaching a point because the acceleration commands were too aggressive. Waypoint commands from the flight controller were not capable of constraining the system’s velocity or acceleration, so the best solution to this problem was implementing an asymptotic approach law. Instead of being directed to the desired location, the drone was sent to a position 1/3 of the way to the intended target. Implementing this adjustment produced calm, accurate, and repeatable trials in which the drone reached the desired setpoint within a set tolerance of 10 cm and began holding position within seconds of arrival.

In summary, trials evaluating the position-control system demonstrated that the system could reliably hover, rotate around the yaw axis, and move to a setpoint while the manipulator was attached. Although this was satisfactory for the position tracking needs of this project, future development could replace the flight computer’s position-controller with a velocity-based controller run on the companion computer, enhancing disturbance rejection and allowing for more direct velocity and acceleration trajectory control.

Landing and Fail-safes

Multiple safety measures were integrated into testing to protect operators and ensure system integrity. All testing was performed inside a drone cage, an enclosed space with padded floors and safety nets to prevent unintended contact with personnel in the event of a runaway vehicle. To ensure continuous supervision of the system, the Jetson companion computer and the Pixhawk flight controller exchange a 10 Hz heartbeat. Loss of this signal or a dead node immediately handed manual control back to the pilot.

If the RC link failed while the pilot was flying manually, the flight controller automatically initiated a slow maneuver back to the home position/takeoff point. If the RC link dropped during autonomous operation, the companion computer detected the interruption and triggered a similar return-to-home routine. This multi-layer fail-safe protocol ensured the drone could safely land in a predetermined position as long as one of the three operation modes was functional.

To prevent human error from causing trial failure or compromising drone integrity, a pre-flight safety checklist was created and the completion of each step was verbally confirmed before liftoff. The complete flying safety checklist is provided in Appendix C

4.2 Detection Model Validation

During the training process, YOLOv11 was configured to log the training process to a service called Weights and Biases (W&B). W&B provided an online interface to monitor the change in training and validation loss over time, as well as other relevant metrics, such as precision ($\frac{\text{\# of correct predictions}}{\text{total \# of predictions}} * 100\%$) and recall ($\frac{\text{\# of objects detected}}{\text{total \# of objects present}} * 100\%$) live throughout the training process. During training on CaNET, YOLOv11 achieved a max precision of 97.0% and a max recall of 89.3% (Figure 59). However, simulation performance did not always match real-world performance because overtraining and differences between synthetic data and the real-world may inflate the validation metrics.

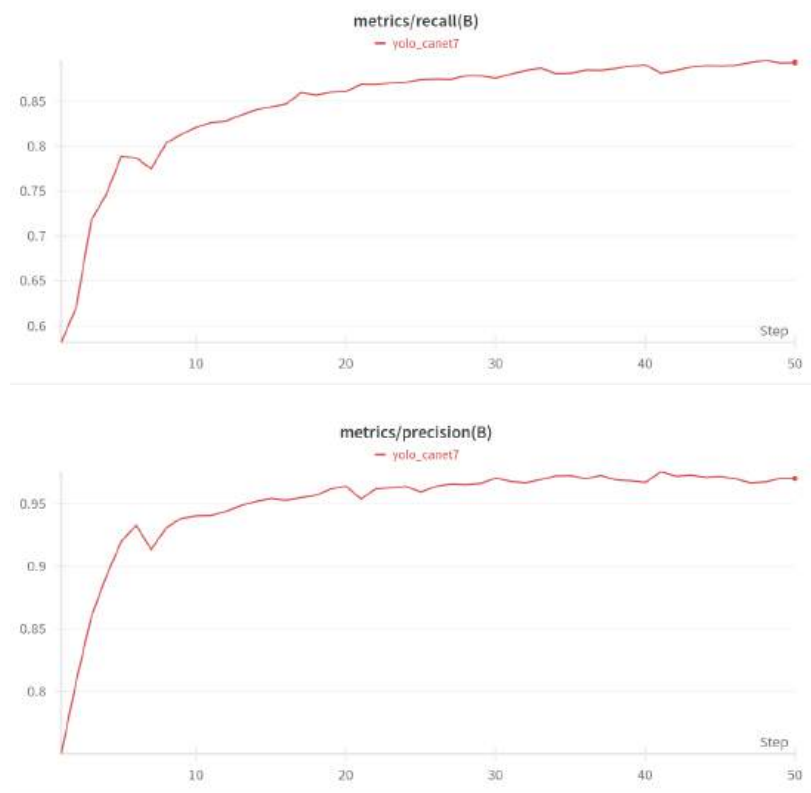


Figure 59: CaNET training graphs from W&B

To measure real-world performance of the trained model, seven videos were recorded. These videos mimicked the drone's point of view at various points in flight and included shots from different heights, fields of view, and locations. Despite all integration testing taking place indoors, the model's performance was evaluated on footage from inside the drone cage and outdoors for better testing of the model's robustness and invariance. These videos were recorded with a 1440p resolution at a rate of 60 fps and downsampled to 640x480 prior to processing. The full video, not just individual frames, was passed into YOLO, resulting in

Measurement	Count/Percentage
Frames Counted	104
Total Num Boxes	262
Total Num Cans	358
Total Correct Cans Detected	218
Total Recall	60.9%
Total False Pos Rate	16.8%
Total Precision	83.2%
Missed Can Rate	39.1%

Table 7: Final Detection Model Metrics

videos annotated with visible model predictions. To quantify the model’s real world success, 15 frames were randomly selected from each video using Python and the number bounding boxes, cans, and correct predictions in each frame was manually recorded. Selecting random frames from each video produced a sample that was representative of the video contents without requiring large amounts of manual data processing. The recall and precision values for real-world performance were calculated using the total number bounding boxes, cans, and correct predictions across the 104 sampled frames and can be seen in Table 7. A small bug in the python script caused a singular frame to be thrown out, decreasing the total number of frames analyzed from 105 to 104.

The full analyzed videos can be found in Appendix D, and a sample from one of the videos can be seen in Figure 60.

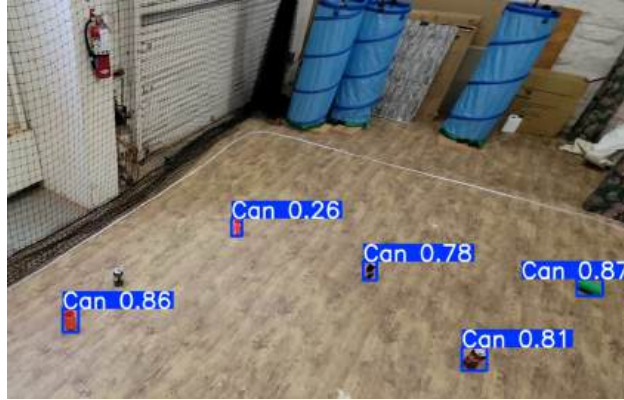


Figure 60: A screenshot from *washburn_1x_processed.mp4* video, with five correct detections and two missed cans.

From qualitative analysis, the model struggled in three main areas: detecting highly reflective objects, identifying non-can cylindrical objects, and detecting partially occluded objects. These difficulties stemmed from problems with the dataset, which included excessive color-jitter (which makes the color and lighting of a given can in the generated dataset random), only using three can models instead of a wider range of shapes & sizes, and not including models of objects other than cans. These limitations of the dataset caused the model to act as a cylinder detector rather than a can detector, identifying objects such as fire extinguishers, light fixtures, and speakers as cans. Sparse regions of high contrast were also seen as cans in some lighting (fallen leaves on gray bricks, patch of green grass surrounded by dirt). While the synthetic dataset did include can models with reflective surfaces, the location of the light source limited reflectivity present in the generated images, especially with smaller cans. This caused the model to frequently miss cans that reflected light towards the camera in real-world scenarios with bright lighting. Another case where the model frequently failed to detect cans present in the image was cases of high occlusion. When part of a can was obscured by a non-can item in the environment, the model will often fail to recognize the can if less than half of it was covered. This was addressed and slightly improved in second iteration of CaNET by rendering

basic shapes to occlude the cans, but the model still struggled in this regard.

Of the metrics used to evaluate the vision model, precision was considered the most important. While recall was also extremely important, the consequences of the model misidentifying an object as a can were much more severe than the consequences of missing a can. When a can was missed the system simply continued looking for a can until one was found, but if an object was misidentified the system would approach it and attempt to grasp it as if it was a can. This scenario required the pilot to quickly seize manual control of the drone to prevent a collision. Because of this, an additional post-processing filter was added to decrease the false-positive rate. During analysis of the real-world data (using the same seven videos mentioned earlier), it was noted that true positives had a mean confidence of 63.4% with a standard deviation of 12.3%, while false positives had a mean confidence of 32.6% with a standard deviation of 25.4%. Filtering the model output and removing all positives with confidence below 50%, allows roughly 75% of the false positives to be ignored and theoretically improves the model’s precision to 95.8%. While this also theoretically drops recall to 52.5%, it did not noticeably worsen the system’s ability to identify cans. Although implementing this filter improved detection accuracy, it did not address the fact that false positives were caused by limitations of the dataset. Ultimately the best way to improve the model performance moving forward would be modifying the dataset to account for the lighting differences, reflectivity, can-like objects, and occlusion the model may encounter in the real world.

4.3 VBM Evaluation

The VBM pipeline can be broken down into two main components, cluster extraction and optimal grasp generation. During cluster extraction, the program identified the (x, y, z) centroid of an object in the world frame by integrating a 2D image output from the detection model with input from a 3D camera. These coordinates were processed into a series of waypoints that the drone followed to smoothly approach a position where the can was within the manipulator’s workspace. To prevent the can from drifting out of the camera’s field of view during the navigation procedure, the 2D to 3D point extraction pipeline (discussed in Section 3.5) and the optimal grasp generation needed to run at the same rate as the camera’s fps.

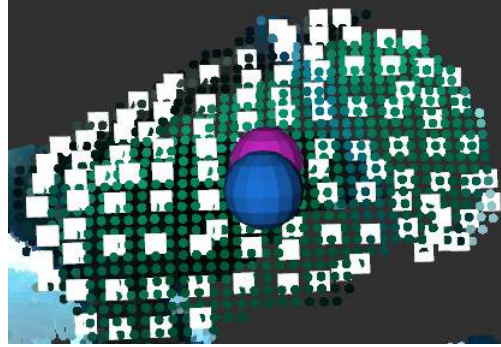
Before evaluating the software performance, the Intel RealSense camera and on-board Jetson computer were tested to determine the maximum reliable frame rate and resolution they could process in real time. Starting at the camera’s max rated values of 1280x720 at 30 fps for 2D and 1280x720 at 90 fps for stereo, the resolution and frame rate of the 2D and stereo images were gradually decreased until the Jetson could reliably render both images simultaneously. After experimentation the optimal rates were 15 fps at 1280x720 resolution for the 2D images and 5 fps at 480x270 resolution for the depth images. When higher frame rates or resolutions were attempted, the program crashed often and began to drop frames after several seconds. From this experimentation, the target processing time for the 2D to 3D point conversion and grasp generation was 0.0667 seconds (i.e., 15 fps).

Once the desired processing speed and resolution were established, the VBM pipeline’s performance relative to this metric was evaluated. The performance of the cluster extraction component was quantified by feeding the system a simulated 2D centroid from the detection model and recording how long the process took. These tests concluded that the extract cluster program consistently and reliably performed the conversion from the 2D point to the 3D point within 0.001 seconds, but took between 0.25 to 0.5 seconds to segment the object cluster from the point cloud. During laboratory testing, the delay caused by segmentation was often longer than the entire trajectory flight time, much too long for the drone control to update its movement to the right location.

However, as shown in Figure 61, the 3D point extracted from the 2D to 3D conversion was comparable to the one extracted from the object cluster. Because of this, the drone could still reliably move to a position where the object could be grabbed. Therefore, to meet the hardware limitations, a modified approach that ignored the object cluster and only relied on the 2D to 3D conversion was used in the VBM pipeline.

Similar testing for the optimal grasp program revealed that generating the ideal grasp for a given cluster took up to an additional 0.5 seconds. Although the pose from this program was more accurate, it was too slow to be integrated into the final pipeline. Additionally, complex grasp analysis was not always beneficial as the limited maneuverability of the 3DOF manipulator meant it could not always perform the ideal grasp.

These findings highlight the importance of balancing perception accuracy with real-time performance in VBM. While the full object segmentation and optimal grasp calculation offered more precise localization and



(a) Front view of the point cloud in RViz



(b) Side view of the point cloud in RViz

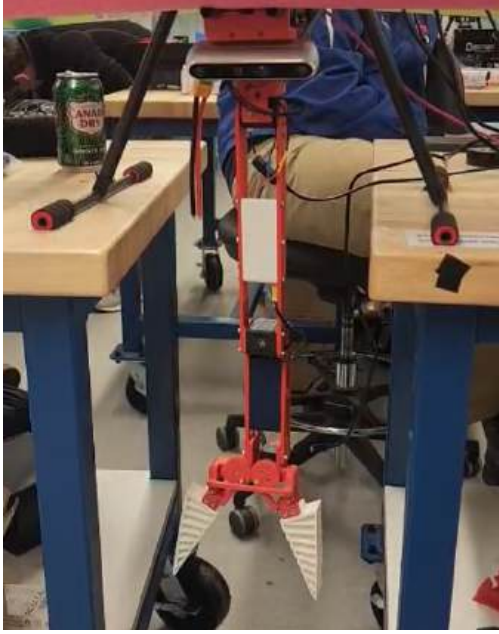
Figure 61: The camera was pointed at a green soda can, and the detection program sent 2D coordinates to the extract cluster program for conversion to 3D coordinates. The white points represent the PCL filtered extracted cluster, the purple sphere is the centroid of this cluster, and the blue sphere is the result of basic 2D to 3D conversion without PCL filtering. In this test, the extraction took 0.45 seconds, while the basic conversion only took 0.001 seconds. The points appear close to each other, with the extracted cluster centroid being slightly more accurate in terms of depth data (relatively further back) and more robust to the 2D point potentially being not centered on the object.

grasp planning information, their processing delays proved too long for smooth and responsive drone and manipulator behavior. In contrast, despite being slightly less accurate, 2D to 3D point conversion allowed the drone to navigate effectively toward targets and allowed the manipulator to grasp objects. This tradeoff demonstrated that in real-time applications, especially ones with limited onboard processing power, efficiency outweighs precision. The streamlined VBM pipeline ultimately enabled a more timely coordination between perception and motion, which was essential for this project to function autonomously.

4.4 Arm Performance

The independent performance of the manipulator was evaluated using static bench testing. Three performance metrics were evaluated, payload capacity, movement speed, and precision. Securely attaching a can to the end effector using an adhesive facilitated testing of the arm's maximum payload without being limited by the gripper's constraints. During testing, a payload was attached to the end effector and incrementally increased until the arm could no longer move from the zero position to the maximum horizontal extension. These positions are shown in Figures 62a and 62b. The failure point of each joint was recorded, which determined the exact joint currently limiting the arm's performance, creating a clear path for future work.

Bench testing was conducted by incrementally increasing the payload on the end effector until the arm began to experience structural deformation or actuator errors. For the entire arm, the payload limit was found to be 250 g. When the payload weight was increased past this point, the third actuator, present in the elbow joint, would experience a torque overload error and stop functioning. This joint being the limiting factor was not surprising, as it was actuated with an XL430 Dynamixel, which has barely $\frac{1}{4}$ of the stall torque of the XM430 used to actuate the shoulder lift joint. Due to the arm's joint configuration, the elbow joint was still supported by the mechanical structure of the arm and would remain in a mostly horizontal position after failure. Because of this, the shoulder lift joint's max payload could be evaluated past the failure of the elbow joint. During this evaluation, the base joint was able to move up to 560 g from the zero position to full forward extension, but failed to maintain the position for an extended period of time. While it could not move the arm to this position, it could also support a load of 1100 g for a few seconds if it was applied once the arm was already in the extended position. A payload higher than this was not



(a) Arm in the zero position during bench testing



(b) Arm extended fully forward during flight. This was the most common orientation that objects were grasped in during testing.

tested due to concerns with the structural integrity of the connection between the shoulder pan joint and the drone frame. Overall, while the arm can theoretically handle manipulating a 560 g payload, it cannot do this without overloading the elbow joint and it also cannot maintain this load for an extended period of time. Based on this the realistic load of the arm was concluded to be 250 g. This capacity is lower than the project specification of 400 g, but changes such as replacing the elbow motor and integrating additional motion planning to lower the sustained torque on the elbow joint could be implemented in future designs to increase the payload limit.

Another aspect of the arm's performance evaluated during bench testing was the time it took to complete the stow/unstow routine, shown in Figures 58 and 57. The stow routine, a five-step procedure, took an average of 7 seconds to complete, with each step taking 1.4 seconds. This was slightly slower than the targeted time of 1.2 seconds for each step trajectory, which was likely caused by the motor being sent the set point of where it should currently be throughout the trajectory. This point did not account for the accumulation of positional error during movement, leading to longer trajectory times as the motor corrected this error. For other movements, such as grasping, motion speed was tuned differently, prioritizing responsiveness and minimizing motor torques to maintain stability and prevent motion in the overall aerial platform. For rapid movements to grasping positions, the arm generally moved from the home position, shown in Figure 58, to the grasping point in 0.5 seconds. This motion could be completed much quicker than the stow routine without disrupting the drone's motion because the actuator movements were small, mostly compensating for the drone's positional error.

The final metric, arm precision, was evaluated by measuring the difference between a known target location and the actual end effector position after motion execution. Testing showed that without a load, the end-effector had an average positional error of 1 cm, which met the accuracy requirement of the project scope. This error was mostly in the Z axis as compliance in the manipulator mounting system, mainly in the shock absorption rings and base joint mount, made determining the exact position of the base joint extremely challenging. Furthermore, when the manipulator was under load, bending in the arm links caused additional Z-axis error, despite the motors being in the desired position. Under a 250 g load, this caused the positional error to increase from 1 cm to 7.5 cm. While this was a much larger positional error, it did not have a large impact on system integration performance as precision is mainly important during grasping. Additionally, this error could be easily reduced by replacing the XL430 motors with XM430s because their more efficient internal gearing and reduced backlash would remove a significant amount of error.

In addition to bench testing, a simulation of the manipulator’s actual workspace compared to the default ZYY workspace was generated. The actual workspace was slightly smaller than the base ZYY workspace as the presence of the drone’s legs limited its range of motion, as seen in Figure 63. Within 40 degrees of the drone’s X axis, the motion of the shoulder lift and elbow joints was completely unrestricted. Outside that range, the manipulator was likely to collide with the drone legs. While there were certain positions that the arm could theoretically reach above or below the landing skids, located 16.4 cm below the base plate, the inverse kinematic calculations used to create the motion trajectories did not account for collisions. As these positions were unreachable in the current configuration, they were not considered part of the manipulator’s actual workspace. This portion of the theoretical workspace was also not easy to retrieve an item from, so expanding the workspace was not considered a priority during development. Overall, despite the boundaries imposed by the legs, the manipulator’s workspace still encompasses the majority of the depth camera’s FOV, making it suitable for the project needs.

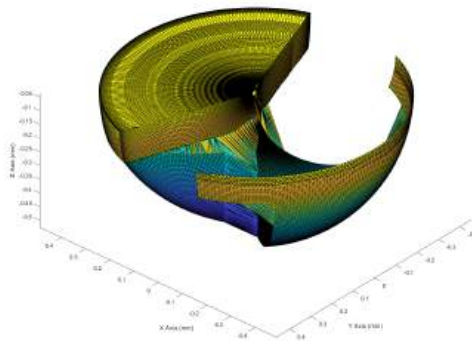


Figure 63: Real workspace of arm acknowledging collisions with legs

This analysis showed that although it did not meet the minimum desired payload capacity, the arm is mechanically capable of retrieving typical roadside litter under 250 g. While it can manipulate payloads up to 560 g through its full workspace, it cannot do this for an extended period of time due to torque limits at the elbow joint and concerns with the mounting integrity of the shoulder pan joint. The precision of the end-effector location is generally sufficient for the project scope, especially when paired with a compliant gripper, but increases greatly under load due to flexibility in the system and unbalanced torques in the joints. The workspace, while slightly limited by drone leg collisions, covers the critical region within the camera’s field of view needed for accurate grasping. Overall, the arm meets core design goals for controlled, bench-top manipulation but would need to be adapted to lift higher payloads and have a farther max reach prior to reliable integration with aerial operations. Replacing all the motors with Dynamixel XM430s, or an even stronger motor, would be a simple, major improvement to the current system.

4.5 Gripper Performance

The gripper’s individual performance was evaluated by operating the actuator independently through the Dynamixel Wizard. The arm was placed in a supported position, ensuring that the gripper’s evaluation would not be affected by the limits of the arm joints. Three criteria were considered when evaluating the gripper: the gripper stroke, the max weight the gripper could hold when stationary, and the max weight the gripper could hold when in motion. The max base to base span of the gripper was 10.5 cm, while the max tip-to-tip span was 29 cm. The minimum span was 0 cm as the tips of the gripper were able to conform to each other at the optimal closed position. This span far exceeded the minimum specification of 5-7 cm and the gripper’s inherent compliance due to the Fin Ray effect meant that the mechanical structure of the gripper did not prevent it from grasping any of the cans in the project scope.

The gripper’s max load was measured in two different ways, the actuation strength and the mechanical strength. Actuation strength was defined by the gripper’s max capacity while the motor was under a load.

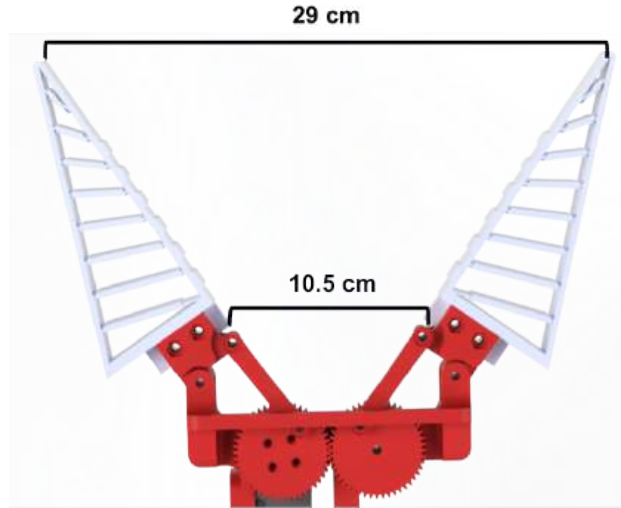


Figure 64: Gripper span visualization

Mechanical strength was defined by the gripper's max capacity when the motor was not powered and the gripper was manually moved to the 'lock position', shown in Figure 65, the maximum closed position. In this position an object could be held in place purely by the gripper's geometry, without the actuators supplying any torque to the mechanism. The maximum actuation strength was measured by incrementally increasing the weight of a hollow, 3D-printed can by placing small weights inside of it until the gripper could no longer support its weight. Three failure points were identified, first when the can began to slip when the arm was violently shaken, second when the arm was exposed to gentle motion, like the kind you would find during UAV flight, and finally when the gripper failed to hold the can in a rigid, horizontal position for at least 10 seconds. During these trials, the max weight under violent motion was found to be 66 g, the max weight under gentle motion was 164 g, and the max weight in a stable position was 287 g. The results of these trials are shown in Table 8.

Weight (g)	Held for 10 seconds without slippage	Held during gentle motion	Held during violent motion
14 g	✓	✓	✓
33 g	✓	✓	✓
43 g	✓	✓	✓
55 g	✓	✓	✓
66 g	✓	✓	✓
100 g	✓	✓	×
133 g	✓	✓	×
164 g	✓	✓	×
205 g	✓	×	×
258 g	✓	×	×
287 g	✓	×	×
319 g	×	×	×

Table 8: Actuated Gripper Evaluation

To establish how much the actuator choice was limiting the gripper's weight limit, the mechanical strength of the gripper was also determined. This was done in the opposite way, where a full water bottle weighing approximately 1.2 kg was attempted to be grasped. After each grasp failed, the weight of the water bottle was incrementally lowered until a grasp was successful. This occurred when the water bottle weighed 985 g,

which marked the max weight the gripper is mechanically able to hold. While carrying this weight, the manipulator could be moved both gradually and violently without the bottle slipping out of the grasp.



Figure 65: When in the 'lock position', the gripper can conform around larger, heavier objects, such as a JBL speaker, without slippage.

The difference between the max mechanical load of the gripper and the max actuated gripper load shows that the gripper was extremely underactuated. Replacing the actuator with a stronger model could increase the gripper's actuated strength up to the max mechanical load. It is important to note that due to the Fin Ray effect, the amount of torque required from the gripper actuator is not a function of the weight of the object being grasped, but rather a function of how much the gripper must deform to reach the 'lock position'. For cylindrical objects, deformation force directly correlates to the object's diameter. A wider object, such as a 12 oz soda can compared to an 8 oz Red Bull can, requires more torque to reach the lock position, but lifting an empty can versus a full can of the same size requires approximately the same amount of torque (ignoring the slight deformation of the empty can under stress). While the max actuated load was much lower than the max mechanical load, this was because the current actuator was not able to reach the lock position. Because of this, when an object reached a certain weight it was able to slightly deform the Fin Ray fingers and use that extra space to slip out.

However, the actuator's max position was extremely close to the 'lock position'. Because of this, even a slightly stronger motor could increase the actuated gripper load up to the max mechanical load. A more in-depth analysis of the Fin Ray finger's behavior under load could determine the exact torque required to deform the gripper to the lock position when grasping different objects. Additionally, the max mechanical load of the gripper is currently limited by the lack of friction between the gripper fingers and the object being grasped. Coating the fingers with a layer of high-friction, compliant material, such as a rubber spray, could increase both the gripper's mechanical and actuated grasp load without an adverse effect on the gripper's deformation ability.

4.6 Integrated System Evaluation

Vision and Manipulator Subsystem

To evaluate the robustness and reliability of the vision-manipulator pipeline, a series of experiments were conducted on the Jetson using the Intel RealSense camera. These tests determined the system's tracking per-

formance, precision, and reachability within the camera’s field of view (FOV) while the drone was positioned statically on a table with the camera and manipulator attached. A can was moved to various positions within the camera’s FOV to test the boundaries of both the manipulator and camera. The system demonstrated reliable tracking performance across most of the viewable area that intersected with the workspace of the manipulator. The arm could successfully reach objects positioned up to 0.5 m away from the front of the drone, up to 0.5 m beneath the arm mount on the drone, and up to 0.25 m to either side of the arm mount. However, the object needed to be at least 0.25 m away from the camera for the VBM program to properly determine the distance to the object. This is because the RealSense camera is based on binocular vision and relies on the overlapping regions of two cameras on the module, as illustrated in Figure 66, and therefore the sensor has a minimum depth distance [21].

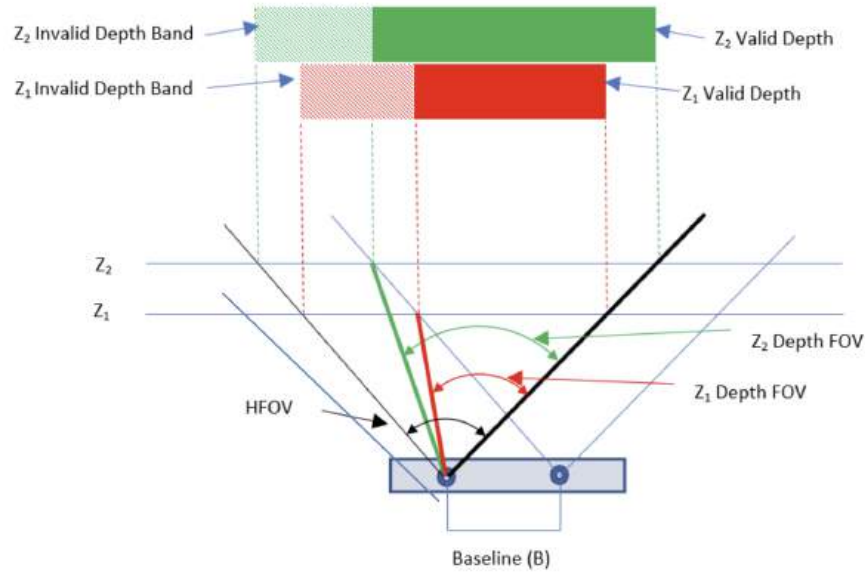


Figure 66: The depth FOV of the stereo camera depends on the overlapping sight regions of the lenses [21], which limits the closest point at which an object’s distance can be tracked by the vision system.

Further testing involved tracking a moving can, both when the object moved slowly across the camera frame and when the can moved faster and experienced vibrations similar to in-flight disturbances. The vision system was generally capable of maintaining centroid tracking under these dynamic conditions, although faster movements often resulted in temporary loss of detection or 3D localization. The arm was able to follow the moving object, but sometimes experienced a short (~ 0.5 second) delay. This was most apparent when the can was moved rapidly across the frame. For example, if the can was moved from one end of the camera’s FOV to the other, the arm took approximately 2 seconds to complete the move.

From this testing, it was concluded that the perception and manipulation pipeline was robust enough to handle the variable conditions the drone could experience during flight. The system also had a reasonable reaction to object movement and could move the manipulator to track and grasp the object with the speed and accuracy needed for in-flight testing. This robustness is essential for future work involving outdoor trash tracking and removal with the drone.

Vision and Drone Subsystem

The experimental setup of the vision-drone integration started with a manual takeoff, followed by autonomous hovering. If hovering was stable, a command was sent to the drone, directing it to spin in place until the onboard vision system located the target can. Once a can was detected, visual servoing, a form of sensor-actuator fusion that relies on computer vision to control the drone, was used to control the drone’s orientation and position with respect to the can. This strategy succeeded in many trials: the drone would detect the can, rotate smoothly to face it, then aligns itself relative to the object’s 3D position in preparation for the

approach maneuver. This continuous feedback loop between the camera input and flight controller ensured a reliable approach in indoor testing conditions. While more than 30 experimental trials were carried out, only 19 of these trials were recorded for detailed performance evaluation. The analysis focused on evaluating three successive autonomous behaviors: maintaining a stable hover with minimal position drift or oscillation, rotating in place to align with the target object, and maneuvering into a grasp-ready position (pictured in Figure 68). Trials where the drone failed at an initial step did not count as failures for future steps. Instead, the number of trials for that behavior was adjusted to reflect how many times it was actually attempted in Table 9.

Behavior	Successes	Trials	Step Success Rate (%)	Total (%)
Drone hovers in place	15	19	78.9	78.9
Drone searches for and locks onto an object	12	15	80.0	63.1
Drone approaches grasp-ready position 67	10	14	71.4	52.6

Table 9: Final visual-servoing evaluation



Figure 67: Drone position after approaching the object, ready to begin attempted to grasp.

The system successfully performed all three behaviors sequentially in 10 of the 19 trials, yielding a 52.6% success rate. This outcome suggests that visual servoing is a reliable in controlled conditions, but remains sensitive to environmental disturbances and visual clutter. Additionally, each successive step introduces more failure points, such as position drift or false positives from the detection model. This cumulation of error causes the total success rate for consecutive steps to be much lower than that step’s individual success rate.

Full Integration

Following the approach step detailed in Section 4.6, the drone attempted to use the manipulator to track the objects position prior to grasping. For safety reasons, this step was not evaluated in the full pipeline before the approach step was confirmed to be functional. As a result, the track and grasp state was only enabled in six of the ten trials that successfully completed the approach step.

Out of nineteen integration trials, just four progressed through to the final step in the pipeline – despite high per-step success rates as indicated in Table 10. Because there was some amount of error present in each step, the error would accumulate, causing fewer trials to make it to the later steps. Due to the payload constraints of the manipulator, no trials were able to successfully complete the entire procedure. In practice,

Behavior	Successes	Trials	Step Success Rate (%)	Total (%)
Drone hovers in place	15	19	78.9	78.9
Drone searches for and locks onto an object	12	15	80.0	63.1
Drone approaches grasp-ready position 67	10	14	71.4	52.6
Manipulator attempts to track and grasp in air	6	10	71.4	31.6
Successful track and grasp attempt 68	4	6	66.7	21.1
Retrieval of object to home position	0	4	0.0	0.0

Table 10: Full system testing



Figure 68: Grasp configuration showing the drone maintaining position during manipulator alignment.

this means that although individual components performed well in isolation, the reliability of the integrated workflow must be considerably improved before deployment of the prototype.

Issues Integration failures were caused by a wide range of hardware and software issues, such as insufficient drone descent speed or momentary detection loss. During real-world testing, even small software errors could manifest as a larger integration failures. For example, momentary detection loss could cause the manipulator to lose contact with the can just before attempting to grasp or cause the drone to lock onto and approach a false positive, such as a fire extinguisher.

In three integration trials, the flight controller overly relied on barometric pressure when calculating its altitude, which caused the drone to drift upwards instead of holding its position. When this happened, an operator took control of the drone and performed a manual landing. While this error caused multiple trials to fail, it was easily fixed by restarting the flight controller. Since this was a sporadic error that did not prevent the system from being tested, a better solution was not explored. However, it was theorized that the error occurred when the flight controller lost confidence in the accuracy of the IMU’s position tracking following multiple consecutive flight tests or a crash landing.

Furthermore, the detection model occasionally misidentified items – such as a portable speaker or wall-mounted OSHA fire extinguishers – as cans. When a false positive was detected, the drone began the approach step, flying towards the misidentified object. If the operator did not take control in time, this could lead to a crash, causing major damage to the drone. The severe consequences of false-positive errors highlights the importance of implementing a more robust object filtering and classification model before full deployment.

Other times, due to downdraft, the lightweight can was be blown away during the approach. In these scenarios, the drone would chase the can until the operator took control to prevent a collision with the safety net. When testing full system integration, two trials failed when the can was blown away by the drone’s updraft. To prevent this issue from occurring in future trials, a 125 g payload, the minimum weight



Figure 69: Post-landing orientation occasionally deviated from the nominal attitude alignment.

needed for stability, was placed inside the can. The weight was also used to simulate a partially-full can as the project scope covered both empty and full cans, a range of 15-400 g. Unfortunately, the drone tended to grasp the top of an object, creating a less stable grip which would cause the object to slip, despite its weighing less than the manipulator's max payload. This problem was the primary reason that prevented integration testing of the final 'deposit' step. Additional reasons included the drone's motion impacting the integrity of the grasp and the drone itself not having enough thrust to lift an object, which occurred in a trial with significantly more weight (~ 500 g).

During several trials, the emergency kill switch had to be used to keep the drone from becoming entangled in the safety net, often producing atypical landings (see Figure 69). Aside from one incident – where delayed activation of the kill switch led to a motor burnout – all other controlled crash landings occurred without loss or significant damage to any system components.

Overall integration was successful at searching for, navigating to, and grasping cans. However, the accumulation of minor faults from across the system meant that completion of the full integration pipeline was not successful. Further testing is needed to continue to address small errors within the individual subsystems and make the entire system more robust.

5 Limitations and Implications

The prototype drone system has shown that using autonomous aerial manipulation for trash removal is feasible. Its perception subsystem reliably identified trash objects, provided the navigation data required for the vehicle to approach targets, and enabled the manipulator to grasp them. Extensive testing - both of individual components and of the fully integrated system - showed the prototype's overall reliability. Nevertheless, several key improvements should be made in future iterations of this project.

First, the flight control algorithms do not take into account the manipulator dynamics or turbulent flight conditions near highway edges. In the current implementation, the flight computer's internal quadcopter control proved robust enough to accommodate slow manipulator motions to prove the prototype feasibility, but it does not compensate for the manipulator in different positions, varying payloads, or higher-speed movements. Although outdoor hover tests under mild gusts confirmed baseline performance, the more intense and rapidly changing wind profiles along edges of a highway median will challenge stability. Adopting a more sophisticated control scheme - e.g., a full model-predictive controller - could provide real-time dynamic compensation during navigation and hovering.

Second, the system's hazard assessment and collision-avoidance capabilities are non-existent. During all tests, a human operator remained ready to take manual control if a collision was imminent or if the manipulator risked striking the ground. By integrating enhanced software checks within the Task Manager node - adding additional cameras and using the depth camera for external obstacle detection - the drone could autonomously mitigate collision risks to both itself and bystanders.

Third, the vision system requires greater robustness and object permanence. The detection model, trained only on undamaged soda cans, occasionally misclassified other cylindrical objects as trash and failed to recognize other trash objects or even crushed cans. Expanding the training dataset to include a broader variety of waste types, deformed objects, and more varied backgrounds would allow for a more robust perception pipeline. Moreover, the current logic reports only the single highest-confidence detection, which can lead to erratic target switching.

Fourth, the VBM pipeline exceeded the targeted latency for point-cloud extraction and optimal-grasp computation. While these techniques have potential for greater accuracy than basic 2D-to-3D conversion and offer valuable data for manipulators with higher degrees of freedom, they must operate in real time to interface effectively with the flight controller and manipulator actuation. Achieving the required processing speed may be possible by dynamically adjusting the PCL filtering parameters according to the Task Manager node current state and the object's location.

Finally, the current search strategy instructs the drone to perform an in-place yaw sweep until a trash object is detected. While effective in confined indoor environments, this method would be inadequate over larger outdoor areas. A more encompassing search pattern - combining rotational sweeps with zig-zag waypoint-based area coverage - should be implemented to expand detection range in highway-median trash collection scenarios.

6 Conclusion

As the demand for single use materials such as plastic wrappers, Styrofoam cups, and soda cans has increased over the past 60 years, roadside litter gradually became a severe problem in the United States. Trash on the sides of roads, highways, and waterways has an adverse impact on road safety and nearby ecological zones. Currently, trash on highway medians is removed by teams of volunteers or prison laborers, an ineffective method that endangers workers and drivers and risks exposing workers to various diseases and illnesses. To address the safety and ethical concerns associated with existing disposal methods, the team developed an aerial manipulator capable of detecting and collecting soda cans. By proving the feasibility of this prototype, the team hopes to promote future development of trash-collecting aerial manipulators and ultimately provide a safe, ethical, and cost-effective way for state governments to monitor and remove litter from public roadsides.

While the scope of this project was limited to an indoor environment collecting a single category of trash, the results and system analysis from this prototype proves that this is an integral path for future research and development. Additionally, the open-source documentation compiled for this project – including a comprehensive list of purchased components, annotated code base, system-architecture diagrams, and the design parameters and considerations presented in this paper – provide a solid foundation for future researchers to build upon.

An aerial manipulator for highway-median trash collection has the potential to complement existing litter-removal efforts across the United States. Future work could focus on refining the current prototype and extending its application to other environments suffering from litter, such as waterways, hiking trails, and coastal beaches.

7 Appendices

A Open-Source Can Models

All open-source models used can be found attached to this paper on WPI open access repository.

B Arm Inverse Kinematics

The first joint of the IK can simply be solved using the atan2 of the y and x coordinates of the goal

$$\theta_1 = \arctan 2(y, x)$$

The second joint of the IK is a bit trickier relying on two intermediate steps α & β as well fixed joint offset $\phi \approx .07$. This is calculated along the RZ plane where $r = \sqrt{x^2 + y^2}$ as shown in Figure 70a.

$$\cos \alpha = \frac{z}{\sqrt{r^2 + z^2}}$$

$$\sin \alpha = \pm \sqrt{1 - \cos^2 \alpha}$$

$$\alpha = \arctan 2(\sin \alpha, \cos \alpha)$$

$$\cos \beta = \frac{L_2^2 + z^2 + r^2 - L_3^2}{2(L_2 + \sqrt{r^2 + z^2})}$$

$$\sin \beta = \pm \sqrt{1 - \cos^2 \beta}$$

$$\beta = \arctan 2(\sin \beta, \cos \beta)$$

$$\theta_2 = \alpha - \beta - \phi$$

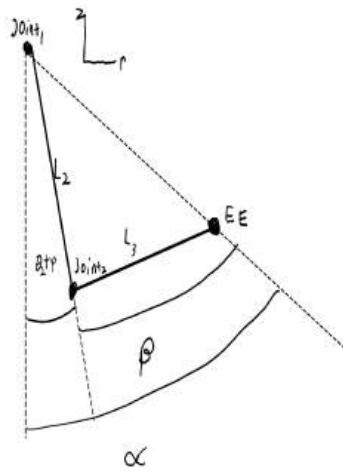
The third joint follows similar to the second using the law of cosines solving for an internal angle γ that opposes θ_3 as shown in Figure 70b.

$$\cos \gamma = \frac{L_2^2 + -L_3^2 - z^2 - r^2}{2(L_2 + L_3)}$$

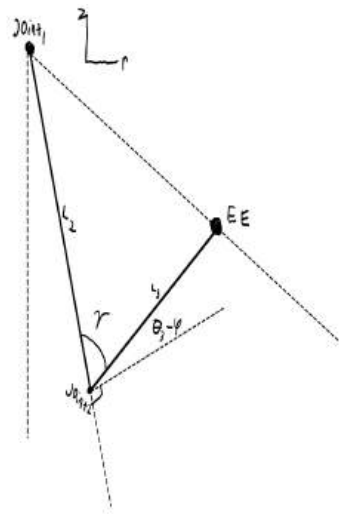
$$\sin \gamma = \pm \sqrt{1 - \cos^2 \gamma}$$

$$\gamma = \arctan 2(\sin \gamma, \cos \gamma)$$

$$\theta_3 = \pi/2 - \gamma + \phi$$



(a) Drawing of Joint2 IK calculations



(b) Drawing of Joint2 IK calculations

Figure 70: Drawings of IK calculations

C Drone Flying Safety Checklist

JOISIE TERRAWARDEN MQP DRONE PRE-FLIGHT CHECKLIST

PRE-FLIGHT INSPECTION

- Check wind conditions suitable for flight.
- Deploy and lock drone arms.
- Ensure battery is secured but NOT connected.
- Check propellers: tight, correct rotation for upright flight corresponding to printed CW or CCW labels.
- Check propeller blades: tight until move with resistance, unfolded.
- Lift/tilt drone – verify no loose components.
- Stow manipulator manually.
- Ensure no wires are tangled or obstructing movement.
- Setup video recording equipment.

FLIGHT SYSTEMS AND INSTRUMENTS

- **GPS antenna:** unfold and secure.
- **Confirm calibration** of accelerometers, gyroscopes, magnetometers.
- Verify **Pixhawk ↔ Jetson** data link.
- Ensure **Jetson** has power.
- Check **Jetson ↔ Dynamixel** data link.
- Confirm **Dynamixel** power connection.
- Power ON remote control.
 - o Set all switches to startup position: throttle down, arm off, position hold, offboard off.
 - o Battery level **60%+** minimum.
- Connect **ground-station receiver** to laptop.
 - o Launch **QGroundControl** app.

PRE-ENGINE START CHECKLIST

- **CLEAR AREA – ENSURE SAFETY.**
- **Only one person** may connect the battery.
- Confirm **remote control** is receiving telemetry data.
- Verify **ground station** connection – no errors or arming-preventing messages.
- SSH into Jetson and **launch ROS node before takeoff.**
- Start video recording.
- **Arm manually** on the ground – propellers must spin.
- **Test kill switch.**
- **Takeoff & hover briefly** to confirm stability and position tracking.
- Switch to Offboard mode for autonomous flight.

SAFETY AND EMERGENCY PROCEDURES

- **No intoxication. No stupidity.** Minimum three brain cells.
- **All personnel must wear safety glasses**, hearing protection, gloves (if necessary).
- Kill switch must be assigned and tested.
- Spotter + operator required (FAA-licensed pilot + valid drone registration and Remote ID).
- Check OpenSky app for airspace clearance.
- Indoor testing: Ensure Washburn + EHS + flying cage certification approval.
- In case of electrical short or a large crash: **IMMEDIATELY unplug battery.**
- Battery fire: **Unplug, if possible, throw away from personnel/equipment** into open area.
- Post-crash: **WAIT 30 seconds before approaching the drone.**
- After a crash: **NEVER power on with propellers attached.**

Fly smart. Fly safe. No excuses.

D Results Videos

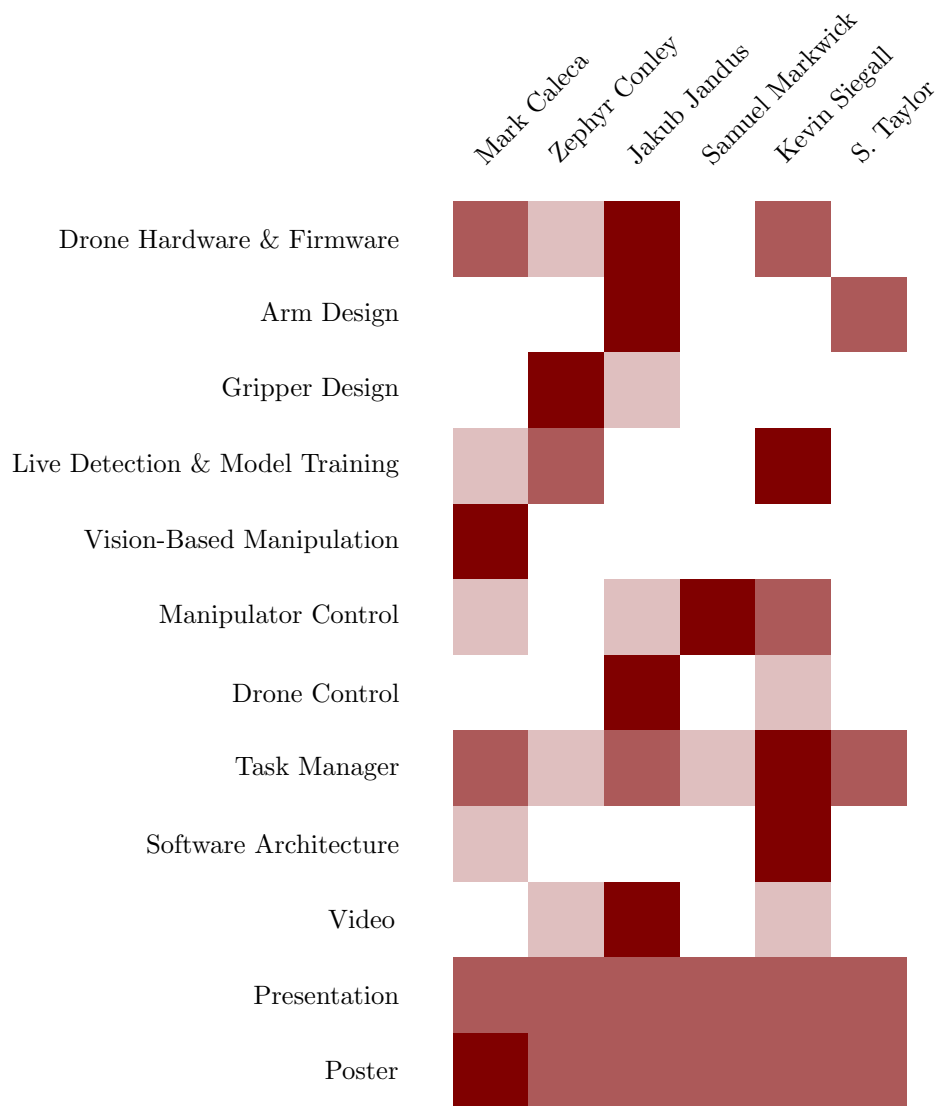
All results videos can be found attached to this paper on WPI open access repository.

E Open-Source Code Repositories

All of our code is open-source and utilizes a GPLv3 license. It can be found here: <https://github.com/Terrawarden-MQP>

F Contributions

Project Contributions



Paper Contributions



Bibliography

- [1] Federal Aviation Administration. *First-Time Pilot Requirements*. URL: https://www.faa.gov/uas/commercial_operators/become_a_drone_pilot.
- [2] Ayham AlAkhras et al. “The Design of a Lightweight Cable Aerial Manipulator with a CoG Compensation Mechanism for Construction Inspection Purposes”. In: *Applied Sciences* 12.3 (Jan. 2022), p. 1173. ISSN: 2076-3417. DOI: 10.3390/app12031173. URL: <https://www.mdpi.com/2076-3417/12/3/1173> (visited on 05/04/2025).
- [3] Gursel Alici. “Softer is Harder: What Differentiates Soft Robotics from Hard Robotics?”. In: *MRS Advances* 3.28 (June 1, 2018), pp. 1557–1568. ISSN: 2059-8521. DOI: 10.1557/adv.2018.159. URL: <https://doi.org/10.1557/adv.2018.159> (visited on 05/03/2025).
- [4] Rodrigo Antunes et al. “Design of Fin Ray Effect Soft Robotic Gripper for Improved Mechanical Performance and Adaptability: Numerical Simulations and Experimental Validation”. In: *2024 20th IEEE/ASME International Conference on Mechatronic and Embedded Systems and Applications (MESA)*. 2024 20th IEEE/ASME International Conference on Mechatronic and Embedded Systems and Applications (MESA). Genova, Italy: IEEE, Sept. 2, 2024, pp. 1–6. ISBN: 9798331516239. DOI: 10.1109/MESA61532.2024.10704855. URL: <https://ieeexplore.ieee.org/document/10704855/> (visited on 01/29/2025).
- [5] Doris Aschenbrenner et al. “Robot-assisted automated sorting techniques for plastic recycling”. In: *Procedia CIRP* 120 (2023), pp. 1232–1237. ISSN: 22128271. DOI: 10.1016/j.procir.2023.09.154. URL: <https://linkinghub.elsevier.com/retrieve/pii/S2212827123008867> (visited on 01/29/2025).
- [6] Avijit Kumar Ashe et al. “Multivariate Data Analysis for Motor Failure Detection and Isolation in A Multicopter UAV Using Real-Flight Attitude Signals”. In: *2023 International Conference on Unmanned Aircraft Systems (ICUAS)*. 2023 International Conference on Unmanned Aircraft Systems (ICUAS). Warsaw, Poland: IEEE, June 6, 2023, pp. 9–16. ISBN: 9798350310375. DOI: 10.1109/ICUAS57906.2023.10155856. URL: <https://ieeexplore.ieee.org/document/10155856/> (visited on 03/20/2025).
- [7] Keep America Beautiful. *Keep America Beautiful 2020 National Litter Study Summary Report*. May 2021. URL: https://kab.org/wp-content/uploads/2021/05/Litter-Study-Summary-Report-May-2021_final_05172021.pdf.
- [8] Keep Massachusetts Beautiful. *Programs to Clean Up and Beautify Massachusetts Highways*. Keep Massachusetts Beautiful. 2024. URL: <https://keepmassbeautiful.org/what-we-do/litter-prevention-cleanup/sponsor-a-highway-program.html> (visited on 10/05/2024).
- [9] *BEBOT: Tahoe’s Beach-Cleaning Robot*. Keep Tahoe Blue. 2025. URL: <https://www.keeptahoeblue.org/combat-pollution/beach-cleaning-robot/> (visited on 10/06/2024).
- [10] Samuel Bello et al. *Beach Swarm - Phase III*. Worcester Polytechnic Institute, 2022.
- [11] Jason Brownlee. *How Much Training Data is Required for Machine Learning?* Machine Learning Mastery. May 23, 2019. URL: <https://machinelearningmastery.com/much-training-data-required-machine-learning/>.
- [12] Mark Caleca et al. “Grasping for Uncertain Trash Removal”. In: *Unpublished manuscript. Robotics Engineering Department, Worcester Polytechnic Institute* (Dec. 2024). URL: <https://drive.google.com/file/d/1RZY6ZQi0T8qfSMPDIKKNX4DP4cAMCfl8/view?usp=sharing>.

- [13] Muqing Cao et al. *AirCrab: A Hybrid Aerial-Ground Manipulator with An Active Wheel*. Mar. 23, 2024. DOI: 10.48550/arXiv.2403.15805. arXiv: 2403.15805[cs]. URL: <http://arxiv.org/abs/2403.15805> (visited on 05/04/2025).
- [14] Rishabh Chadha and Amey Kulkarni. *ros2_torch_trt*. original-date: 2020-11-11T18:57:42Z. 2020. URL: https://github.com/NVIDIA-AI-IOT/ros2_torch_trt (visited on 04/10/2025).
- [15] Xiaoyu Chen et al. *Understanding Domain Randomization for Sim-to-real Transfer*. Mar. 13, 2022. DOI: 10.48550/arXiv.2110.03239. arXiv: 2110.03239. URL: <http://arxiv.org/abs/2110.03239> (visited on 04/10/2025).
- [16] Matthew Clarke. “ACLU Report Details Exploitation of Prisoners for Forced Labor”. In: *Prison Legal News* (May 2023), p. 42. URL: <https://www.prisonlegalnews.org/news/2023/may/1/aclu-report-details-exploitation-prisoners-forced-labor/>.
- [17] Qingzheng Cong, Wen Fan, and Dandan Zhang. “TacFR-Gripper: A Reconfigurable Fin-Ray-Based Gripper with Tactile Skin for In-Hand Manipulation”. In: *Actuators* 13.12 (Dec. 17, 2024), p. 521. ISSN: 2076-0825. DOI: 10.3390/act13120521. URL: <https://www.mdpi.com/2076-0825/13/12/521> (visited on 01/29/2025).
- [18] Wikipedia contributors. *Swashplate (aeronautics)*. In: Wikipedia, The Free Encyclopedia., Mar. 15, 2025. URL: [https://en.wikipedia.org/w/index.php?title=Swashplate_\(aeronautics\)&oldid=1280608267](https://en.wikipedia.org/w/index.php?title=Swashplate_(aeronautics)&oldid=1280608267) (visited on 03/20/2025).
- [19] Peter Corke. *Inverse Kinematics for a 2-Joint Robot Arm Using Geometry — Robot Academy*. Inverse Kinematics for a 2-Joint Robot Arm Using Geometry. 2019. URL: <https://robotacademy.net.au/lesson/inverse-kinematics-for-a-2-joint-robot-arm-using-geometry/> (visited on 05/03/2025).
- [20] Intel Corporation. *Intel RealSense Depth Camera D455*. Intel RealSense. URL: <https://www.intelrealsense.com/depth-camera-d455/> (visited on 01/16/2025).
- [21] Intel Corporation. *Intel RealSense Product Family D400 Series Datasheet*. Datasheet 337029-013. Mar. 2023. URL: <https://www.intelrealsense.com/wp-content/uploads/2023/03/Intel-RealSense-D400-Series-Datasheet-March-2023.pdf> (visited on 04/22/2025).
- [22] iRobot Corporation. *About iRobot*. iRobot History. 2024. URL: <https://about.irobot.com/history> (visited on 10/06/2024).
- [23] Whitney Crooks et al. “Fin Ray® Effect Inspired Soft Robotic Gripper: From the RoboSoft Grand Challenge toward Optimization”. In: *Frontiers in Robotics and AI* 3 (Nov. 10, 2016). ISSN: 2296-9144. DOI: 10.3389/frobt.2016.00070. URL: <http://journal.frontiersin.org/article/10.3389/frobt.2016.00070/full> (visited on 05/03/2025).
- [24] J. Denavit and R. S. Hartenberg. “A Kinematic Notation for Lower-Pair Mechanisms Based on Matrices”. In: *Journal of Applied Mechanics* 22.2 (June 1, 1955), pp. 215–221. ISSN: 0021-8936, 1528-9036. DOI: 10.1115/1.4011045. URL: <https://asmedigitalcollection.asme.org/appliedmechanics/article/22/2/215/1110292/A-Kinematic-Notation-for-Lower-Pair-Mechanisms> (visited on 05/01/2025).
- [25] Zhifeng Deng and Miao Li. “Learning Optimal Fin-Ray Finger Design for Soft Grasping”. In: *Frontiers in Robotics and AI* 7 (Feb. 12, 2021), p. 590076. ISSN: 2296-9144. DOI: 10.3389/frobt.2020.590076. URL: <https://www.frontiersin.org/articles/10.3389/frobt.2020.590076/full> (visited on 01/29/2025).
- [26] Huixu Dong et al. “Grasp analysis and optimal design of robotic fingertip for two tendon-driven fingers”. In: *Mechanism and Machine Theory* 130 (Dec. 2018), pp. 447–462. ISSN: 0094114X. DOI: 10.1016/j.mechmachtheory.2018.08.028. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0094114X18312412> (visited on 01/29/2025).
- [27] Sergey Dorodnicov. *The basics of stereo depth vision*. Intel RealSense, July 17, 2018. URL: <https://www.intelrealsense.com/stereo-depth-vision-basics/> (visited on 01/16/2025).

- [28] Grant Eden. “Using a UAV and Edge Computing to Identify and Throw Away Trash”. Masters Thesis. Pennsylvania State University, Mar. 2020. URL: https://etda.libraries.psu.edu/files/final_submissions/21404.
- [29] JOHNATHON P. EHSANI, JEFFREY P. MICHAEL, and ELLEN J. MacKENZIE. “The Future of Road Safety: Challenges and Opportunities”. In: *The Milbank Quarterly* 101 (Suppl 1 Apr. 2023), pp. 613–636. ISSN: 0887-378X. DOI: 10.1111/1468-0009.12644. URL: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC10126980/> (visited on 04/27/2025).
- [30] Alain René Frey 1990-. “Design of an autonomous litter detection and collection system for Icelandic beaches”. Thesis. June 2024. URL: <https://skemman.is/handle/1946/47664> (visited on 03/17/2025).
- [31] Jake Gray. *Prison Labor: Modern-Day Slavery under the Thirteenth Amendment*. Columbia Undergraduate Law Review. Jan. 6, 2019. URL: <https://www.culawreview.org/journal/prison-labor-modern-day-slavery-under-the-thirteenth-amendment> (visited on 10/05/2024).
- [32] IMARC Group. *Robotic Vacuum Cleaner Market Size, Share, Trends and Forecast by Type, Type of Charging, Distribution Channel, Application, End User, and Region, 2025-2033*. 5936396. Feb. 2025. URL: <https://www.researchandmarkets.com/reports/5936396/robotic-vacuum-cleaner-market-size-share> (visited on 03/20/2025).
- [33] Kaiming He et al. *Deep Residual Learning for Image Recognition*. Dec. 10, 2015. DOI: 10.48550/arXiv.1512.03385. arXiv: 1512.03385. URL: <http://arxiv.org/abs/1512.03385> (visited on 03/17/2025).
- [34] M. Honarpardaz et al. “Finger design automation for industrial robot grippers: A review”. In: *Robotics and Autonomous Systems* 87 (Jan. 2017), pp. 104–119. ISSN: 09218890. DOI: 10.1016/j.robot.2016.10.003. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0921889015303171> (visited on 01/29/2025).
- [35] Asmaul Hosna et al. “Transfer learning: a friendly introduction”. In: *Journal of Big Data* 9.1 (Oct. 22, 2022), p. 102. ISSN: 2196-1115. DOI: 10.1186/s40537-022-00652-w. URL: <https://doi.org/10.1186/s40537-022-00652-w> (visited on 03/17/2025).
- [36] Roshan Kumar Hota, Aditya Negi, and C. S. Kumar. “Vision-Based Grasp Planning Based on Grasp Quality Metrics and Its Hardware Implementation”. In: *Mechanism and Machine Science*. Ed. by Dibakar Sen, Santhakumar Mohan, and Gondi Kondaiah Ananthasuresh. Series Title: Lecture Notes in Mechanical Engineering. Singapore: Springer Singapore, 2021, pp. 377–390. ISBN: 9789811544767 9789811544774. DOI: 10.1007/978-981-15-4477-4_26. URL: http://link.springer.com/10.1007/978-981-15-4477-4_26 (visited on 10/08/2024).
- [37] Moad Idrissi, Mohammad Salami, and Fawaz Annaz. “A Review of Quadrotor Unmanned Aerial Vehicles: Applications, Architectural Design and Control Algorithms”. In: *Journal of Intelligent & Robotic Systems* 104.2 (Feb. 2022), p. 22. ISSN: 0921-0296, 1573-0409. DOI: 10.1007/s10846-021-01527-7. URL: <https://link.springer.com/10.1007/s10846-021-01527-7> (visited on 03/17/2025).
- [38] “Insurance Institute for Highway Safety: Stopped-Vehicle Crashes Result in Hundreds of Fatalities Per Year”. In: *Targeted News Service; Washington, D.C.* (June 4, 2021). URL: <http://ezproxy.wpi.edu/login?url=https://www.proquest.com/wire-feeds/insurance-institute-highway-safety-stopped/docview/2537109182/se-2?accountid=29120>.
- [39] Glenn Jocher and Jing Qiu. *Ultralytics YOLO11*. Version 11.0.0. 2024. URL: <https://github.com/ultralytics/ultralytics>.
- [40] K Bouzgou et al. “A New Classification and Aerial Manipulation Q-PRR Design”. In: (2019). Publisher: Unpublished. DOI: 10.13140/RG.2.2.24863.79520. URL: <http://rgdoi.net/10.13140/RG.2.2.24863.79520> (visited on 09/16/2024).
- [41] Kasra Karimi and Ardeshir Faghri. “The Issues of Roadside Litter: A Review Paper”. In: *Current Urban Studies* 9.4 (Oct. 29, 2021), pp. 779–803. DOI: 10.4236/cus.2021.94046. URL: <https://www.scirp.org/journal/paperinformation?paperid=113864> (visited on 03/16/2025).

- [42] Kai Sherng Khor, Chao Liu, and Chien Chern Cheah. “Robotic Grasping of Unknown Objects Based on Deep Learning-Based Feature Detection”. In: *Sensors* 24.15 (July 26, 2024), p. 4861. ISSN: 1424-8220. DOI: 10.3390/s24154861. URL: <https://www.mdpi.com/1424-8220/24/15/4861> (visited on 05/01/2025).
- [43] Danilo Klasanović et al. “Application and Optimal Design of a Soft Robotic Gripper for Grasping Objects of Arbitrary Shape”. In: *2024 11th International Conference on Electrical, Electronic and Computing Engineering (IcETTRAN)*. 2024 11th International Conference on Electrical, Electronic and Computing Engineering (IcETTRAN). Nis, Serbia: IEEE, June 3, 2024, pp. 1–6. ISBN: 9798350386998. DOI: 10.1109/IcETTRAN62308.2024.10645079. URL: <https://ieeexplore.ieee.org/document/10645079/> (visited on 01/29/2025).
- [44] Marek Kraft et al. “Autonomous, Onboard Vision-Based Trash and Litter Detection in Low Altitude Aerial Images Collected by an Unmanned Aerial Vehicle”. In: *Remote Sensing* 13.5 (Mar. 4, 2021), p. 965. ISSN: 2072-4292. DOI: 10.3390/rs13050965. URL: <https://www.mdpi.com/2072-4292/13/5/965> (visited on 03/17/2025).
- [45] RamMohan Reddy Kundavaram et al. “Advances in Autonomous Robotics for Environmental Cleanup and Hazardous Waste Management”. In: *Asia Pacific Journal of Energy and Environment* 12.1 (Feb. 4, 2025), pp. 1–16. ISSN: 2312-282X, 2312-2005. DOI: 10.18034/apjee.v12i1.788. URL: <https://i-proclaim.my/journals/index.php/apjee/article/view/788> (visited on 03/17/2025).
- [46] Dasom Lee, David J. Hess, and Michiel A. Heldeweg. “Safety and privacy regulations for unmanned aerial vehicles: A multiple comparative analysis”. In: *Technology in Society* 71 (Nov. 2022), p. 102079. ISSN: 0160791X. DOI: 10.1016/j.techsoc.2022.102079. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0160791X22002202> (visited on 03/20/2025).
- [47] Yanghao Li et al. *Benchmarking Detection Transfer Learning with Vision Transformers*. Nov. 22, 2021. DOI: 10.48550/arXiv.2111.11429. arXiv: 2111.11429. URL: <http://arxiv.org/abs/2111.11429> (visited on 03/17/2025).
- [48] Tsung-Yi Lin et al. *Microsoft COCO: Common Objects in Context*. Feb. 21, 2015. DOI: 10.48550/arXiv.1405.0312. arXiv: 1405.0312. URL: <http://arxiv.org/abs/1405.0312> (visited on 03/17/2025).
- [49] Kevin M. Lynch and Frank C. Park. *Modern robotics: mechanics, planning, and control*. Cambridge: Cambridge university press, 2017. ISBN: 9781107156302.
- [50] Anqi Mao, Mehryar Mohri, and Yutao Zhong. *Cross-Entropy Loss Functions: Theoretical Analysis and Applications*. June 20, 2023. DOI: 10.48550/arXiv.2304.07288. arXiv: 2304.07288. URL: <http://arxiv.org/abs/2304.07288> (visited on 03/17/2025).
- [51] Jonathan Masinick, Hualiang Teng, and Nesley Orochena. “The Impact of Rubbernecking on Urban Freeway Traffic”. In: *Journal of Transportation Technologies* 4.1 (Jan. 15, 2014), pp. 116–125. DOI: 10.4236/jtts.2014.41012. URL: <https://www.scirp.org/journal/paperinformation?paperid=42488> (visited on 03/16/2025).
- [52] Md. Jahid Hasan. “Design and Fabrication of an Adaptive Gripper with Fin-Ray Effect”. In: (2022). DOI: 10.13140/RG.2.2.31566.88649. URL: <https://rgdoi.net/10.13140/RG.2.2.31566.88649> (visited on 01/29/2025).
- [53] Syed Agha Hassnain Mohsan et al. “Towards the Unmanned Aerial Vehicles (UAVs): A Comprehensive Review”. In: *Drones* 6.6 (June 15, 2022). Publisher: MDPI AG, p. 147. ISSN: 2504-446X. DOI: 10.3390/drones6060147. URL: <https://www.mdpi.com/2504-446X/6/6/147> (visited on 05/01/2025).
- [54] Douglas Morrison, Peter Corke, and Jürgen Leitner. *Closing the Loop for Robotic Grasping: A Real-time, Generative Grasp Synthesis Approach*. Version Number: 2. 2018. DOI: 10.48550/ARXIV.1804.05172. URL: <https://arxiv.org/abs/1804.05172> (visited on 10/08/2024).
- [55] Sabhari Natarajan, Galen Brown, and Berk Calli. “Aiding Grasp Synthesis for Novel Objects Using Heuristic-Based and Data-Driven Active Vision Methods”. In: *Frontiers in Robotics and AI* 8 (July 15, 2021), p. 696587. ISSN: 2296-9144. DOI: 10.3389/frobt.2021.696587. URL: <https://www.frontiersin.org/articles/10.3389/frobt.2021.696587/full> (visited on 05/01/2025).

- [56] Keiron O'Shea and Ryan Nash. *An Introduction to Convolutional Neural Networks*. Dec. 2, 2015. DOI: 10.48550/arXiv.1511.08458. arXiv: 1511.08458. URL: <http://arxiv.org/abs/1511.08458> (visited on 03/17/2025).
- [57] Aníbal Ollero. "Aerial Robotic Manipulators". In: *Encyclopedia of Robotics*. Ed. by Marcelo H Ang, Oussama Khatib, and Bruno Siciliano. Berlin, Heidelberg: Springer, 2019, pp. 1–8. ISBN: 9783642416101. DOI: 10.1007/978-3-642-41610-1_78-1. URL: https://doi.org/10.1007/978-3-642-41610-1_78-1 (visited on 03/21/2025).
- [58] Anibal Ollero et al. "Past, Present, and Future of Aerial Robotic Manipulators". In: *IEEE Transactions on Robotics* 38.1 (Feb. 2022), pp. 626–645. ISSN: 1552-3098, 1941-0468. DOI: 10.1109/TR0.2021.3084395. URL: <https://ieeexplore.ieee.org/document/9462539/> (visited on 10/05/2024).
- [59] World Health Organization. *Road traffic injuries*. Road traffic injuries. Dec. 13, 2023. URL: <https://www.who.int/news-room/fact-sheets/detail/road-traffic-injuries> (visited on 04/27/2025).
- [60] Jason Pacheco. "CSC380: Principles of Data Science Introduction to Machine Learning". University of Arizona. URL: http://www.pachecoj.com/courses/csc380_fall21/lectures/mlintro.pdf.
- [61] Janis Peksa and Dmytro Mamchur. "A Review on the State of the Art in Copter Drones and Flight Control Systems". In: *Sensors* 24.11 (May 23, 2024), p. 3349. ISSN: 1424-8220. DOI: 10.3390/s24113349. URL: <https://www.mdpi.com/1424-8220/24/11/3349> (visited on 03/20/2025).
- [62] Erwin Prassler et al. "A Short History of Cleaning Robots". In: *Autonomous Robots* 9.3 (Dec. 1, 2000), pp. 211–226. ISSN: 1573-7527. DOI: 10.1023/A:1008974515925. URL: <https://doi.org/10.1023/A:1008974515925> (visited on 10/05/2024).
- [63] Pedro F. Proença and Pedro Simões. *TACO: Trash Annotations in Context for Litter Detection*. Mar. 17, 2020. DOI: 10.48550/arXiv.2003.06975. arXiv: 2003.06975[cs]. URL: <http://arxiv.org/abs/2003.06975> (visited on 10/09/2024).
- [64] *PX4*. 2025. URL: <https://px4.io/>.
- [65] *PX4 ROS2 Bridge*. 2025. URL: https://docs.px4.io/main/en/middleware/uxrce_dds.html.
- [66] Joseph Redmon et al. *You Only Look Once: Unified, Real-Time Object Detection*. May 9, 2016. DOI: 10.48550/arXiv.1506.02640. arXiv: 1506.02640. URL: <http://arxiv.org/abs/1506.02640> (visited on 04/10/2025).
- [67] *Refactoring Guru: Observer*. URL: <https://refactoring.guru/design-patterns/observer>.
- [68] Hamid Rezatofighi et al. *Generalized Intersection over Union: A Metric and A Loss for Bounding Box Regression*. Apr. 15, 2019. DOI: 10.48550/arXiv.1902.09630. arXiv: 1902.09630. URL: <http://arxiv.org/abs/1902.09630> (visited on 03/17/2025).
- [69] Máximo A. Roa and Raúl Suárez. "Grasp quality measures: review and performance". In: *Autonomous Robots* 38.1 (Jan. 2015), pp. 65–88. ISSN: 0929-5593, 1573-7527. DOI: 10.1007/s10514-014-9402-3. URL: <http://link.springer.com/10.1007/s10514-014-9402-3> (visited on 10/08/2024).
- [70] *Roadside Design Guide*. Repr., 4. ed. Washington: AASHTO, 2012. ISBN: 9781560515098.
- [71] Fabio Ruggiero, Vincenzo Lippiello, and Anibal Ollero. "Aerial Manipulation: A Literature Review". In: *IEEE Robotics and Automation Letters* 3.3 (July 2018), pp. 1957–1964. ISSN: 2377-3766, 2377-3774. DOI: 10.1109/LRA.2018.2808541. URL: <https://ieeexplore.ieee.org/document/8299552/> (visited on 03/17/2025).
- [72] Radu Bogdan Rusu and Steve Cousins. "3D is here: Point Cloud Library (PCL)". In: *2011 IEEE International Conference on Robotics and Automation*. 2011 IEEE International Conference on Robotics and Automation (ICRA). Shanghai, China: IEEE, May 2011, pp. 1–4. ISBN: 978-1-61284-386-5 978-1-61284-385-8. DOI: 10.1109/ICRA.2011.5980567. URL: <https://ieeexplore.ieee.org/document/5980567/> (visited on 01/17/2025).
- [73] Ji Hyeon Shin et al. "A Universal Soft Gripper with the Optimized Fin Ray Finger". In: *International Journal of Precision Engineering and Manufacturing-Green Technology* 8.3 (May 2021), pp. 889–899. ISSN: 2288-6206, 2198-0810. DOI: 10.1007/s40684-021-00348-1. URL: <https://link.springer.com/10.1007/s40684-021-00348-1> (visited on 01/29/2025).

- [74] Bruno Siciliano. *Springer Handbook of Robotics*. In collab. with Oussama Khatib. 2nd ed. Springer Handbooks Ser. Cham: Springer International Publishing AG, 2016. 1 p. ISBN: 978-3-319-32552-1.
- [75] Gidugu Lakshmi Srinivas, Arshad Javed, and Lisa Marie Faller. “Versatile 3D-printed fin-ray effect soft robotic fingers: lightweight optimization and performance analysis”. In: *Journal of the Brazilian Society of Mechanical Sciences and Engineering* 46.6 (May 23, 2024), p. 382. ISSN: 1806-3691. DOI: 10.1007/s40430-024-04957-0. URL: <https://doi.org/10.1007/s40430-024-04957-0> (visited on 05/03/2025).
- [76] State. Refactoring.guru. URL: <https://refactoring.guru/design-patterns/state> (visited on 05/02/2025).
- [77] Matthew Sweeney et al. *Trashbot: Mobile Garbage Collecting Robot*. Worcester Polytechnic Institute, 2024.
- [78] Trombia Technologies. *Trombia Free*. Trombia Technologies. URL: <https://trombia.com/products/trombia-free/> (visited on 03/12/2025).
- [79] Charles W. Thomas. “Prisoners’ Rights and Correctional Privatization: A Legal and Ethical Analysis”. In: *Business & Professional Ethics Journal* 10.1 (1991), pp. 3–45. URL: <http://www.jstor.org/stable/27800833>.
- [80] Ultralytics. *YOLO Performance Metrics*. Ultralytics YOLO Docs. URL: <https://docs.ultralytics.com/guides/yolo-performance-metrics> (visited on 04/07/2025).
- [81] Rohith Venkataramanan et al. “Perception-Driven Robotic Manipulation for Packaging Stack of Envelopes: Gripper Design and Manipulation Strategies”. In: *2024 IEEE International Conference on Advanced Intelligent Mechatronics (AIM)*. 2024 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM). Boston, MA, USA: IEEE, July 15, 2024, pp. 1639–1644. ISBN: 9798350355369. DOI: 10.1109/AIM55361.2024.10637196. URL: <https://ieeexplore.ieee.org/document/10637196/> (visited on 01/29/2025).
- [82] Aswin Warriar and Prashant Minz. “Robotics in Dairy & Food Industry”. In: 9th Convention of India Dairy Engineers. Karnal, Haryana, Sept. 2014.
- [83] What Is Shore A Hardness? - Hongju. <https://hongjusilicone.com/>. URL: <https://hongjusilicone.com/shore-a-hardness/> (visited on 05/03/2025).
- [84] Zhen Xie, Xinquan Liang, and Canale Roberto. “Learning-based robotic grasping: A review”. In: *Frontiers in Robotics and AI* 10 (Apr. 4, 2023), p. 1038658. ISSN: 2296-9144. DOI: 10.3389/frobt.2023.1038658. URL: <https://www.frontiersin.org/articles/10.3389/frobt.2023.1038658/full> (visited on 03/21/2025).
- [85] Xing San et al. “Multiview Image Coding Based on Geometric Prediction”. In: *IEEE Transactions on Circuits and Systems for Video Technology* 17.11 (Nov. 2007), pp. 1536–1548. ISSN: 1051-8215, 1558-2205. DOI: 10.1109/TCSVT.2007.905382. URL: <http://ieeexplore.ieee.org/document/4358677/> (visited on 03/13/2025).
- [86] Quman Xu et al. *ATDM: An Anthropomorphic Aerial Tendon-driven Manipulator with Low-Inertia and High-Stiffness*. May 8, 2024. DOI: 10.48550/arXiv.2405.04821. arXiv: 2405.04821[cs]. URL: <http://arxiv.org/abs/2405.04821> (visited on 05/04/2025).
- [87] Yang Yang et al. “A 3D-Printed Fin Ray Effect Inspired Soft Robotic Gripper with Force Feedback”. In: *Micromachines* 12.10 (Oct. 2021), p. 1141. ISSN: 2072-666X. DOI: 10.3390/mi12101141. URL: <https://www.mdpi.com/2072-666X/12/10/1141> (visited on 01/29/2025).
- [88] Yuxiao Zhou and Kecheng Yang. “Exploring TensorRT to Improve Real-Time Inference for Deep Learning”. In: *2022 IEEE 24th Int Conf on High Performance Computing & Communications; 8th Int Conf on Data Science & Systems; 20th Int Conf on Smart City; 8th Int Conf on Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys)*. 2022 IEEE 24th Int Conf on High Performance Computing & Communications; 8th Int Conf on Data Science & Systems; 20th Int Conf on Smart City; 8th Int Conf on Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys). Dec. 2022, pp. 2011–2018. DOI:

10.1109/HPCC-DSS-SmartCity-DependSys57074.2022.00299. URL: <https://ieeexplore.ieee.org/document/10074837> (visited on 03/17/2025).

- [89] Filip Zoric et al. “Towards Instance Segmentation-Based Litter Collection with Multi-Rotor Aerial Vehicle”. In: *2024 International Conference on Unmanned Aircraft Systems (ICUAS)*. 2024 International Conference on Unmanned Aircraft Systems (ICUAS). Chania - Crete, Greece: IEEE, June 4, 2024, pp. 631–637. ISBN: 9798350357882. DOI: 10.1109/ICUAS60882.2024.10557120. URL: <https://ieeexplore.ieee.org/document/10557120/> (visited on 03/17/2025).